

Laboratórios de Análise de Discurso Político

Francisco Brandão

Lucas M Pavelski

2026-06-03

Table of contents

Introdução	5
Objetivos do curso	6
Motivação	6
Bibliografia	6
Programa detalhado	7
Aula 1 - 06/03/2026:	7
Aula 2 - 20/03/2026:	7
Aula 3 - 10/04/2026:	7
Aula 4 - 15/05/2026:	8
Aula 5 - 24/04/2026:	8
Aula 6 - 22/05/2026:	8
Aula 7 - 29/05/2026:	8
Aula 8 - 26/06/2026:	8
Avaliação	8
 1 Linguagem R	 9
1.1 Programando scripts em R:	9
1.2 Ambientes para programar em R	10
1.2.1 Rstudio	11
1.2.2 Google Colab	11
1.2.3 Console R	11
1.2.4 Scripts R	11
1.2.5 Notebooks R	12
1.3 Comandos básicos	13
1.3.1 Operadores aritméticos	13
1.3.2 Operadores relacionais e lógicos	13
1.3.3 Variáveis	15
1.3.4 Condicionais	18
1.3.5 Funções	19
1.3.6 Vetores	22
1.3.7 Data frames	25
1.3.8 Bibliotecas e pacotes	27
1.3.9 Strings (cadeias de caracteres)	29
1.4 Exemplo: Análise de Sentimentos com R	30
1.5 Para saber mais	33

1.6	Atividade prática	33
2	Palavras, morfemas e tokens	34
2.1	Obtendo dados textuais	34
2.1.1	Tipos de fontes de dados textuais	34
2.1.2	Exemplo: Obtendo Discursos da Câmara dos Deputados	37
2.2	Níveis de análise textual	56
2.3	A estrutura das palavras: morfemas, lemas e radicais	57
2.3.1	Morfemas	57
2.3.2	Stemização (Corte do radical)	59
2.3.3	Caracteres	60
2.4	Tokenização (Dividindo o texto)	60
2.5	Stopwords	64
2.6	Palavras no contexto (Keywords-in-context)	65
2.7	Exemplo: Palavras mais Utilizadas em Discursos	70
2.8	Para saber mais	79
2.9	Atividade prática	80
	Glossário	81
	Introdução	81
	Aula 1	81
	Referências	82
	Appendices	83
	Expressões regulares	83

Introdução

Câmara dos Deputados
Centro de Formação, Treinamento e Aperfeiçoamento
Programa de Pós-Graduação



MESTRADO PROFISSIONAL EM PODER LEGISLATIVO PLANO DE CURSO DE DISCIPLINA

DISCIPLINA: ANÁLISE DO DISCURSO POLÍTICO

Período: 1º semestre 2026

Carga horária total: 30 h/a

Código: MEST.7.09.17

PROFESSORES

E-mail

FRANCISCO BRANDÃO, Dr.

francisco.brandao@camara.leg.br

LUCAS MARCONDES PAVELSKI, Dr.

lucas.pavelski@camara.leg.br

CURRÍCULOS RESUMIDOS

FRANCISCO BRANDÃO, Dr.

Grupo de Pesquisa e Extensão (GPE): Ciência de Dados Aplicada ao Estudo do Poder Legislativo: abordagem computacional e métodos de análise

Doutor em Ciência Política pela Universidade da Califórnia, Santa Barbara (2018), nos campos de Política Comparada, Comunicação Política e Tecnologia da Informação e Sociedade. Possui graduação em Jornalismo pela Universidade de São Paulo (2000) e mestrado em Ciência Política pela Universidade de Brasília (2008). Entre 2022 e 2023, foi pesquisador associado da Universidade de Loughborough, no Reino Unido, no projeto PANCOPOP – Comunicação Populista em Tempos de Pandemia. Jornalista na Diretoria de Comunicação da Câmara dos Deputados.

Currículo Lattes: <http://lattes.cnpq.br/9403525530814720>

LUCAS MARCONDES PAVELSKI, Dr.

Doutor e Mestre em Engenharia Elétrica e Informática Industrial pela Universidade Tecnológica Federal do Paraná (2021) com ênfase em sistemas inteligentes. Possui graduação em Ciência da Computação pela Universidade Estadual do Centro-Oeste do Paraná (2012). Atualmente é Analista de TI na Câmara dos Deputados, atuando na Seção de Soluções para a Área Política, com experiência em Ciência de Dados e Engenharia de Software.

Currículo Lattes: <http://lattes.cnpq.br/9287626771427206>

EMENTA DA DISCIPLINA

História, conceitos importantes, questões teóricas e método da Análise de Discurso (AD). Poder como controle, controle do discurso, estratégias discursivas, discurso e poder. Discurso como produção social. Formações discursivas e formações ideológicas. Relação entre estrutura e agência na produção discursiva. Análise argumentativa; discurso e argumentação. As instâncias midiáticas do discurso político. Relações entre discurso político e discurso midiático. Processos hermenêuticos envolvidos na AD. Narrativas, accounts, justificações, estratégias retóricas e uso de significantes vazios na política e nas instâncias jurídicas e midiáticas.

OBJETIVO GERAL DA DISCIPLINA

O aluno deverá compreender e aplicar empiricamente os conceitos básicos relacionados com as principais teorias de análise do discurso e seu instrumental metodológico.

OBJETIVOS ESPECÍFICOS DA DISCIPLINA

Objetivos do curso

Apresentar um ferramental tecnológico para auxiliar na análise de discursos políticos utilizando técnicas de Processamento de Linguagem Natural (PLN).

Motivação

- O volume de dados textuais disponíveis cresceu exponencialmente com a digitalização da informação e o advento das redes sociais.
- Pesquisa quantitativa complementa a pesquisa qualitativa, permitindo análises em larga escala.
- Ferramentas de PLN permitem extrair insights valiosos de grandes volumes de texto.
- Aplicações práticas em diversas áreas, como ciência política, sociologia, marketing e saúde pública.
- Desenvolvimento de habilidades técnicas em análise de dados textuais, programação e modelagem estatística.
- Foco prático do Mestrado Profissional em Poder Legislativo.

Bibliografia

JURAFSKY, D; MARTIN, J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition, 2025.

Livro-texto com referencial teórico sobre processamento de linguagem natural, linguística computacional e análise de conteúdo (Jurafsky and Martin 2026).

Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>

O'NEILL, L et al. Quantitative discourse analysis at Scale—AI, NLP and the transformer revolution. SoDa Laboratories Working Paper Series, v. 35, n. 11, p. 18-26, 2021.

Artigo apresentando NLP do estado-da-arte aplicada a estudos sociais (O'Neill et al. 2021) .

Disponível em: <https://ideas.repec.org/p/ajr/sodwps/2021-12.html>

SILGE, J.; ROBINSON, D. Text mining with R: a tidy approach. Beijing ; Boston: O'Reilly, 2017.

Livro-texto com referencial teórico e prático sobre mineração de textos com R e biblioteca tidytext (Silge and Robinson 2017).

Disponível em: <https://www.tidytextmining.com/>

WICKHAM, Hadley et al. R for data science. Sebastopol: O'Reilly, 2017.

Livro-texto com referencial prático sobre ciência de dados com R (Wickham 2023).

Disponível em: <https://r4ds.hadley.nz/> (inglês) e <https://pt.r4ds.hadley.nz/> (tradução em português).

CASELI, Helena de Medeiros; NUNES, Maria das Graças Volpe. Processamento de linguagem natural: conceitos, técnicas e aplicações em português. 2024.

Livro-texto com referencial teórico sobre processamento de linguagem natural em português (Caseli and Nunes 2024).

Disponível em: <https://brasileiraspln.com/livro-pln/2a-edicao/>

Programa detalhado

Aula 1 - 06/03/2026:

- Aplicações de PLN em análise de discurso político
- Linguagem R
- Operações básicas com textos em R

Aula 2 - 20/03/2026:

- Obtenção de dados textuais
- Dados do legislativo
- Morfemas, lemas e stemming
- Tokens e stopwords

Aula 3 - 10/04/2026:

- n-gramas
- Collocations
- TF-IDF
- Análise de tópicos

Aula 4 - 15/05/2026:

- Análise sintática
- Embeddings
- Similaridade de Documentos

Aula 5 - 24/04/2026:

- Classificação de textos
- Modelos de linguagem

Aula 6 - 22/05/2026:

- Análise de sentimentos
- Reconhecimento de entidades nomeadas

Aula 7 - 29/05/2026:

- Modelos discursivos
- Retrieval Augmented Generation (RAG)

Aula 8 - 26/06/2026:

- Apresentações finais

Avaliação

A avaliação será feita com base na participação nas aulas e na apresentação final de um projeto prático de análise de discurso político utilizando técnicas de processamento de linguagem natural.

1 Linguagem R

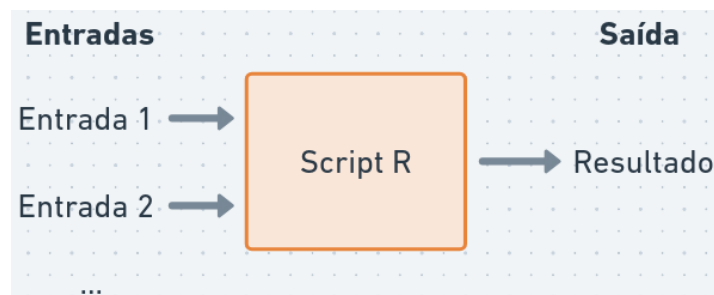
i Nesta aula

- Como utilizar a linguagem R
- Comandos básicos
- Utilizando bibliotecas
- Funções para processamento de texto

Link para Google Colab: <https://colab.research.google.com/drive/1LalxshzmyMmTyfuIaFbp7RYcoP5Qnvuo>

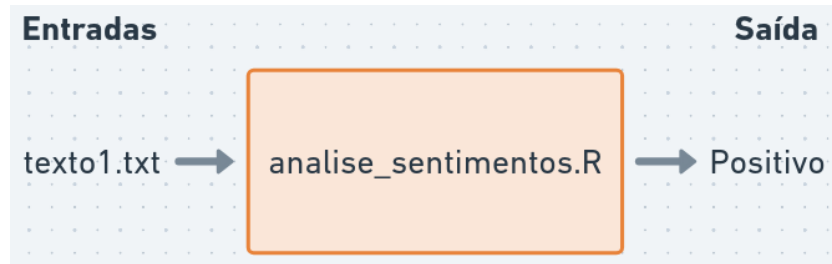
- **R** é uma linguagem de programação e ambiente de software para computação estatística e gráficos
- Criada por Ross Ihaka e Robert Gentleman em 1993
- Muito utilizada em ciência de dados, estatística e análise de texto
- Possui uma grande comunidade de usuários e desenvolvedores, com muitas bibliotecas para análise de textos

1.1 Programando scripts em R:



Script R: sequência de comandos R que podem ser executados para realizar uma tarefa específica (ex: análise de sentimentos)

Em PLN:



`analise_sentimentos.R` é um arquivo contendo instruções para:

- Ler arquivos de texto
- Processar textos (tokenização, remoção de stopwords, stemming)
- Analisar sentimentos (encontrar palavras positivas e negativas)
- Gerar saída (relatório, gráfico, tabela)

O **interpretador R (Rscript)** é um programa que executa os comandos do script e produz a saída desejada. Por exemplo, pelo terminal de comandos:

```
{ $ Rscript analise_sentimentos.R texto1.txt } > "Positivo"
```

1.2 Ambientes para programar em R

O interpretador R está disponível para download em <https://cran-r.c3sl.ufpr.br/>

```
## (código utilizado apenas para exibir páginas no Google Colab)
if (!requireNamespace("htmltools", quietly = TRUE)) {
  install.packages("htmltools")
}
htmltools::tags$iframe(src = "https://cran-r.c3sl.ufpr.br/", width = "100%", height = "315")
```

- No Windows, é necessário também instalar o RTools, disponível em <https://cran.r-project.org/bin/windows/Rtools/>, que inclui ferramentas para auxiliar a instalação de pacotes do R.

Podemos escrever comandos R em diversos ambientes:

1.2.1 Rstudio

- Ambiente de programação integrado (IDE) instalada localmente (Windows, Mac, Linux).
- Possui diversas funcionalidades para facilitar a escrita e execução de código R, como editor de scripts, console interativo, visualização de gráficos e gerenciamento de pacotes.

Disponível em: <https://posit.co/downloads/>.

Uma alternativa é o IDE Positron, disponível em: <https://positron.posit.co/download.html>.

1.2.2 Google Colab

- Ambiente online para programação em Python, mas também suporta R com algumas configurações adicionais (selecione “R” como linguagem de programação para criar um notebook R).
- Permite compartilhar notebooks interativos com código R e resultados.

Disponível em: <https://colab.research.google.com/>.

Outra alternativa é a posit.cloud, disponível em: <https://posit.cloud>.

1.2.3 Console R

- Interface de linha de comando para executar comandos R diretamente no terminal.
- Um comando é executado por vez (digite o comando e aperte Enter), e o resultado é exibido imediatamente.
- É um ambiente menos amigável para desenvolver scripts longos, mas útil para testes rápidos e para o aprendizado da linguagem.

Acessando o console R:

- No terminal, digite R para iniciar o console R e q() para sair.
- No RStudio, o console R está disponível na parte inferior da interface.
- No Google Colab, clique em “Terminal” e digite “R” para iniciar o console R.

1.2.4 Scripts R

- Arquivo de texto com extensão .R que contém uma sequência de comandos R.
- Permite organizar e reutilizar código, facilitando a execução de tarefas complexas.

Como escrever um script R:

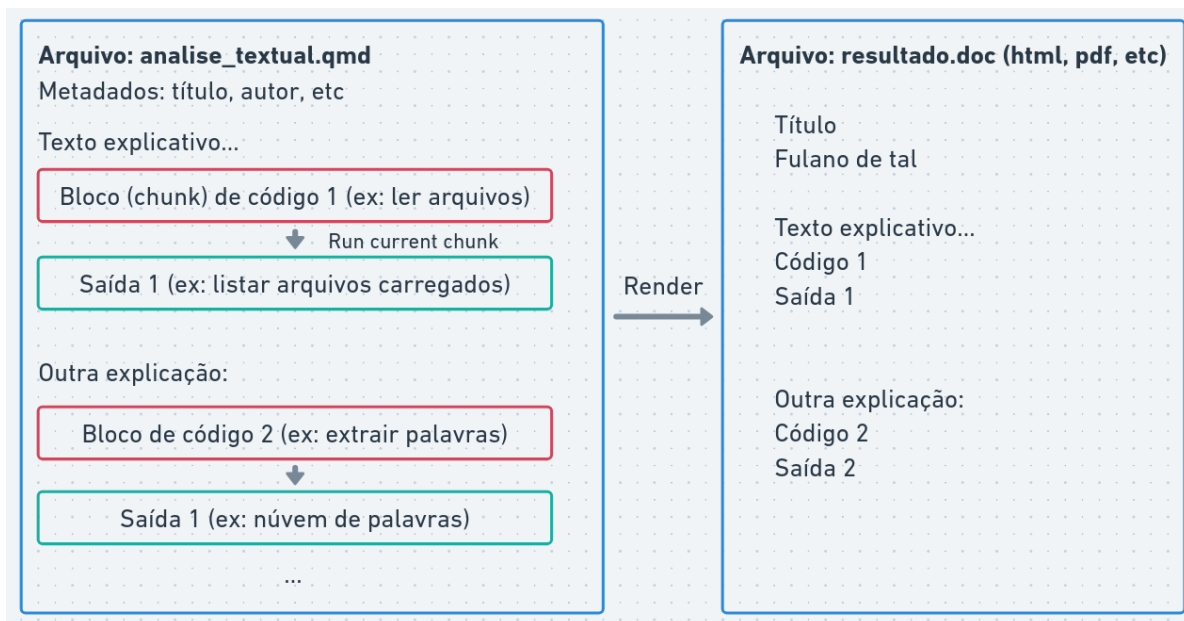
- No RStudio, crie um novo script em: **File > New File > R Script**. Clique em “Source” para executar o script inteiro ou selecione linhas específicas e clique em “Run” para executar apenas essas linhas.

1.2.5 Notebooks R

- Notebooks são documentos que combinam texto explicativo, código R e resultados (gráficos, tabelas) em um formato interativo.
- Permitem criar relatórios dinâmicos e interativos, facilitando a comunicação dos resultados.

Para criar e executar notebooks R, você pode usar o Rstudio ou o Google Colab:

- No Rstudio, crie um novo notebook: **File > New File > Quarto Document**.
- No Google Colab, clique em “File” > “New Notebook” e selecione “R” como linguagem de programação.



Notebooks possuem dois tipos de células:

- células de texto para escrever explicações e comentários
- células de código para escrever e executar códigos em R (o botão “play” executa cada célula individualmente).

1.3 Comandos básicos

- Executar cálculos
- Operadores relacionais e lógicos
- Variáveis
- Listas e vetores
- Funções
- Strings (cadeias de caracteres)

1.3.1 Operadores aritméticos

R pode ser utilizado para cálculos básicos, por exemplo:

```
print(4 + 3)
```

```
[1] 7
```

- `print(*)` é uma função que exibe o resultado de uma expressão na tela.

```
print(5 * 2 + 3)
```

```
[1] 13
```

```
print((6 - 1) / 2)
```

```
[1] 2.5
```

Qualquer código após `#` é considerado um comentário e não é executado:

```
print(8 / 3)      # Isto é um comentário, não será executado
```

```
[1] 2.666667
```

1.3.2 Operadores relacionais e lógicos

Operadores relacionais são utilizados para comparar valores.

Igualdade:

```
print(5 == 5)
```

[1] TRUE

Diferença:

```
print(1 != 2)
```

[1] TRUE

Maior que:

```
print(4 > 3)
```

[1] TRUE

Menor que:

```
print(2 < 5)
```

[1] TRUE

Maior ou igual a:

```
print(3 >= 3)
```

[1] TRUE

- Expressões lógicas: retornam verdadeiro (TRUE) ou falso (FALSE)

Conjunção “e”:

```
print(4 > 3 && 8 > 2)
```

[1] TRUE

- `&&` é o operador lógico “e” (conjunção) que retorna TRUE se ambas as expressões forem verdadeiras, caso contrário retorna FALSE:

- `TRUE && TRUE` retorna `TRUE`
- `TRUE && FALSE` retorna `FALSE`
- `FALSE && TRUE` retorna `FALSE`
- `FALSE && FALSE` retorna `FALSE`

Disjunção “ou”:

```
print(4 > 3 || 2 > 8)
```

[1] `TRUE`

- `||` é o operador lógico “ou” (disjunção), que retorna `TRUE` se pelo menos uma das expressões for verdadeira. Caso contrário, retorna `FALSE`.
- `TRUE || TRUE` retorna `TRUE`
- `TRUE || FALSE` retorna `TRUE`
- `FALSE || TRUE` retorna `TRUE`
- `FALSE || FALSE` retorna `FALSE`

Negação “não”

```
print(!(5 > 3))
```

[1] `FALSE`

- `!` é o operador lógico “não” (negação) que inverte o valor lógico de uma expressão. Se a expressão for `TRUE`, retorna `FALSE`, e se for `FALSE`, retorna `TRUE`.
- `!(TRUE)` retorna `FALSE`
- `!(FALSE)` retorna `TRUE`

```
print((TRUE && FALSE) || TRUE)
```

[1] `TRUE`

1.3.3 Variáveis

Variáveis são utilizadas para guardar valores na memória do computador:

```
a <- 5  
  
print(a)
```

[1] 5

Utilizamos o operador <- para atribuir valores a variáveis.

O valor armazenado pode ser utilizado em outras expressões:

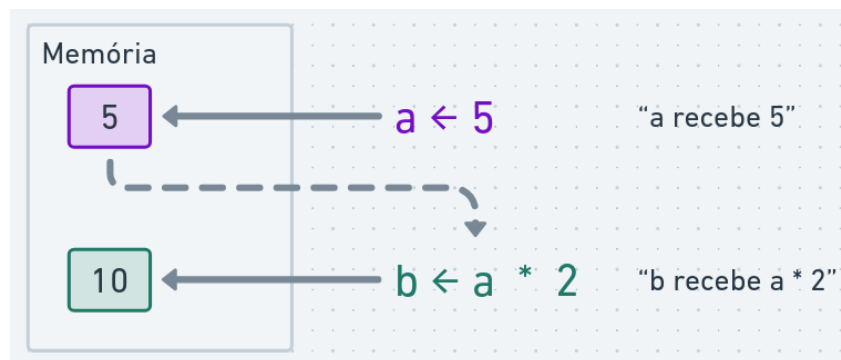
```
print(a * 2 + 3)
```

[1] 13

Ou podemos criar novas variáveis a partir de outras:

```
b <- a * 2  
  
print(b)
```

[1] 10



O valor armazenado na variável pode ser alterado:

```
a <- 10  
  
print(a)
```



```
[1] 10
```

Variáveis podem armazenar diferentes tipos:

```
p <- 1 == 1
q <- 4 < 3

print(p || q)
```

```
[1] TRUE
```

O nome de uma variável deve começar com uma letra e pode conter letras, números (exceto no começo), underscores (_) e pontos:

```
isto_é_uma_variável <- 10
aluno_aprovado_1 <- isto_é_uma_variável == 10

print(isto_é_uma_variável)
```

```
[1] 10
```

```
print(aluno_aprovado_1)
```

```
[1] TRUE
```

Exemplo de cálculo do índice de massa corporal (IMC):

```
peso <- 70 # peso em kg
altura <- 1.75 # altura em metros

imc <- peso / (altura * altura)

imc_saudeavel <- imc >= 18.5 && imc <= 24

print(imc)
```

```
[1] 22.85714
```

```
print(imc_saudavel)
```

```
[1] TRUE
```

1.3.4 Condicionais

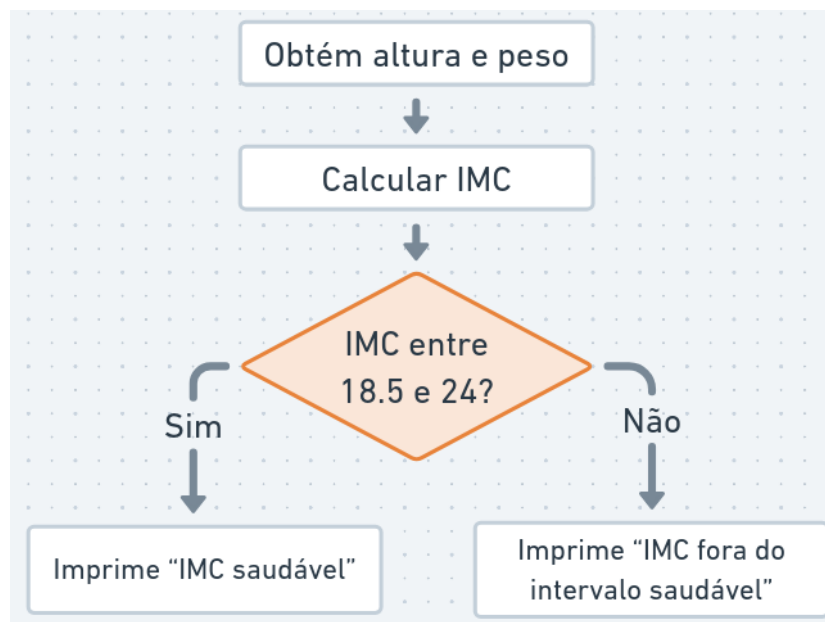
O comando `if` é utilizado para executar um bloco de código apenas se uma condição for verdadeira:

```
x <- 10

if (x > 5) {
  print("x é maior que 5")
}
```

```
[1] "x é maior que 5"
```

O comando `else` pode ser utilizado para executar um bloco de código alternativo caso a condição seja falsa:



```
peso <- 70 # peso em kg
altura <- 1.75 # altura em metros

imc <- peso / (altura * altura)

print(imc)
```

```
[1] 22.85714
```

```
imc_saudavel <- imc >= 18.5 && imc <= 24

if (imc_saudavel) {
  print("IMC saudável")
} else {
  print("IMC fora do intervalo saudável")
}
```

```
[1] "IMC saudável"
```

1.3.5 Funções

Uma **função** é um bloco de código que possui um nome e realiza uma tarefa específica, geralmente para operações comuns que se repetem.

Por exemplo, **print** é a função que usamos para exibir resultados na tela.

R possui várias funções para estatística e análise de dados. Alguns exemplos são:

Valor absoluto:

```
print(abs(-5))
```

```
[1] 5
```

Raiz quadrada:

```
raiz <- sqrt(16)

print(raiz)
```

[1] 4

Logaritmo:

```
log(100, base = 10)
```

[1] 2

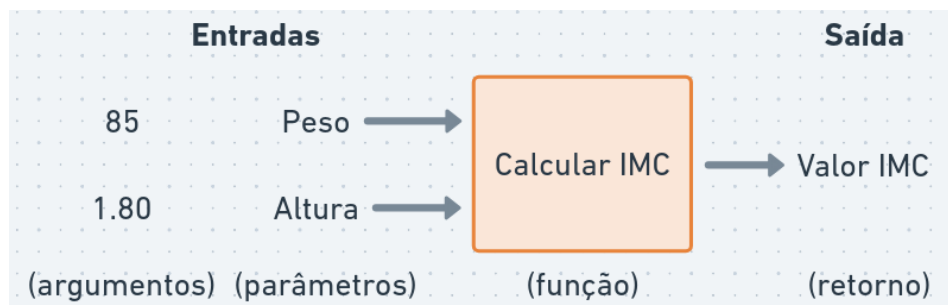
- O número 100 é o primeiro argumento da função.
- O número 10 é o segundo argumento, nomeado como base. Por padrão, a base é o número de Euler ($\exp(1)$), calculando o logaritmo natural.
- Para obter ajuda sobre uma função, execute `help(log)` ou `?log`.

Também é possível definir nossas próprias funções:

```
calcular_imc <- function(peso, altura) {  
  altura_quadrado <- altura * altura  
  return(peso / altura_quadrado)  
}
```

```
imc <- calcular_imc(85, 1.80)  
print(imc)
```

[1] 26.23457



Observe a sintaxe para declarar uma função:

```
nome_da_funcao <- function(argumentos) {  
  # corpo da função  
}
```

A expressão `return(...)` é utilizada para indicar o valor que a função deve retornar. Se `return()` não for especificado, a função retornará o resultado da última expressão avaliada em seu corpo.

Note que qualquer variável criada dentro de uma função é uma variável **local**, ou seja, ela não pode ser acessada fora do escopo da função:

```
print(altura_quadrado)
```

Error in print(altura_quadrado): objeto 'altura_quadrado' não encontrado

Após definir uma função, podemos utilizá-la em qualquer parte do código:

```
teste_imc <- function(peso, altura) {  
  imc <- calcular_imc(peso, altura)  
  imc_saudavel <- imc >= 18.5 && imc <= 24  
  return(imc_saudavel)  
}  
  
print(calcular_imc(80, 1.75) < calcular_imc(70, 1.65))
```

[1] FALSE

Em certos casos, utilizamos o resultado de uma função como argumento de outra função:

```
converter_para_celsius <- function(fahrenheit) {  
  celsius <- (fahrenheit - 32) * 5 / 9  
  return(celsius)  
}  
  
esta_congelando <- function(celsius) {  
  congelando <- celsius <= 0  
  return(congelando)  
}  
  
temperatura_inverno <- 25 # Graus Fahrenheit (aprox. -3.8 °C)  
  
resultado <- esta_congelando(converter_para_celsius(temperatura_inverno))  
  
print(resultado)
```

```
[1] TRUE
```

Em R, podemos utilizar o *operador pipe* (`|>`) para encadear funções de forma mais legível:

```
temperatura_inverno <- 25 # Graus Fahrenheit (aprox. -3.8 °C)

resultado <- temperatura_inverno |>
  converter_para_celsius() |>
  esta_congelando()

print(resultado)
```

```
[1] TRUE
```

- No exemplo, a expressão `temperatura_inverno` é passada como primeiro argumento para a função `converter_para_celsius()`, e o resultado dessa função é então passado como primeiro argumento para a função `esta_congelando()`.

O operador pipe (`|>`) torna o código mais legível, permitindo que as funções sejam encadeadas de maneira clara e fluida.

1.3.6 Vetores

Vetores são estruturas que armazenam uma sequência de elementos do mesmo tipo:

```
vetor_num <- c(1, 2, 3, 4, 5, 6, 7)

print(vetor_num)
```

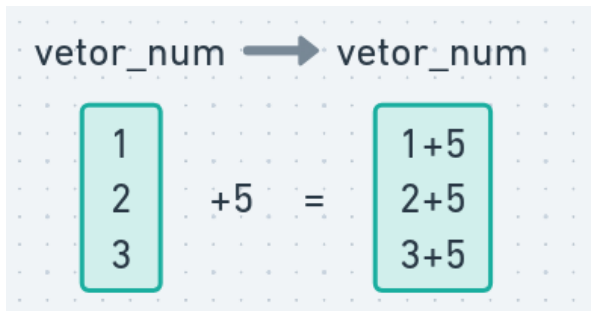
```
[1] 1 2 3 4 5 6 7
```

Operações aritméticas podem ser aplicadas a todos os elementos de um vetor simultaneamente:

```
vetor_num <- c(1, 2, 3)
vetor_num <- vetor_num + 5

print(vetor_num)
```

```
[1] 6 7 8
```



É possível criar sequências numéricas de forma simples:

```
vetor_seq <- 1:10  
  
print(vetor_seq)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Algumas funções operam sobre vetores:

```
print(sum(1:100))
```

```
[1] 5050
```

```
print(mean(1:100))
```

```
[1] 50.5
```

```
print(sd(1:100))
```

```
[1] 29.01149
```

Para acessar um elemento específico do vetor, utilizamos colchetes []:

```
vetor_pares <- c(2, 4, 6, 8, 10)  
  
print(vetor_pares[3])
```

```
[1] 6
```

Para alterar elementos do vetor também utilizamos colchetes:

```
vetor_seq <- 1:10
vetor_seq[3] <- 999

print(vetor_seq)
```

```
[1] 1 2 999 4 5 6 7 8 9 10
```

Podemos acessar múltiplos elementos do vetor:

```
print(vetor_pares[c(1, 4)])
```

```
[1] 2 8
```

Também podemos filtrar elementos do vetor com expressões lógicas:

```
pares_maior_5 <- vetor_pares > 5
elementos_maior_5 <- vetor_pares[pares_maior_5]

print(vetor_pares)
```

```
[1] 2 4 6 8 10
```

```
print(pares_maior_5)
```

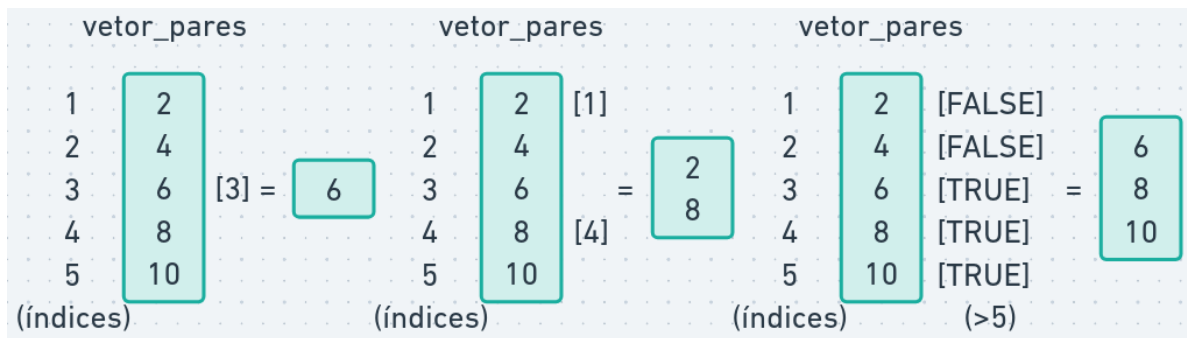
```
[1] FALSE FALSE TRUE TRUE TRUE
```

```
print(elementos_maior_5)
```

```
[1] 6 8 10
```

```
print(vetor_pares[vetor_pares > 5])
```

```
[1] 6 8 10
```

O operador `%in%` é utilizado para verificar se certos elementos estão presentes em um vetor:

```
vetor_a <- c(2, 3, 10)

print(3 %in% vetor_a)
```

```
[1] TRUE
```

```
vetor_b <- c(3, 4, 5, 6, 7)

print(vetor_a %in% vetor_b)
```

```
[1] FALSE TRUE FALSE
```

1.3.7 Data frames

R possui vários tipos de dados, os principais são:

- Numéricos: números inteiros ou decimais (ex: 5, 3.14)
- Lógicos: TRUE ou FALSE
- Strings: texto entre aspas (ex: "Olá, mundo!")

Data frames são estruturas de dados tabulares, onde cada coluna é um vetor e cada linha representa uma observação.

```
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  turma = c("A", "B", "A")
)

print(df)
```

	nome	idade	turma
1	João	30	A
2	Maria	25	B
3	Pedro	28	A

Data frames são amplamente utilizados para análise de dados, permitindo manipular e analisar conjuntos de dados tabulares.

Essencialmente, um data frame é uma lista de vetores de mesmo comprimento, em que cada vetor representa uma coluna. Cada coluna pode conter um tipo diferente de dado (numérico, lógico, texto etc.).

Acessando colunas:

```
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  turma = c("A", "B", "A")
)

print(df$idade)
```

```
[1] 30 25 28
```

```
print(df[, "idade"])
```

```
[1] 30 25 28
```

Acessando linhas:

```
print(df[1, ])
```

	nome	idade	turma
1	João	30	A

Acessando linha e coluna:

```
print(df[1, "idade"])
```

```
[1] 30
```

Para selecionar alunos com mais de 25 anos:

```
alunos_mais_velhos <- df[df$idade > 25, ]  
print(alunos_mais_velhos)
```

```
      nome idade turma  
1  João     30      A  
3 Pedro     28      A
```

1.3.8 Bibliotecas e pacotes

Bibliotecas (ou pacotes) são conjuntos de funções, dados e documentação que estendem as funcionalidades do R. Geralmente são escritas por outros usuários e compartilhadas com a comunidade (ex: `tidyverse`).

Para instalar uma biblioteca, utilizamos o comando `install.packages()`.

```
if (!"tidyverse" %in% installed.packages()) {  
  install.packages("tidyverse")  
}
```

Pesquisadores do mundo todo contribuem com bibliotecas no repositório oficial do R, <https://cran-r.c3sl.ufpr.br/>.

Para acessar as funções de uma biblioteca, é necessário carregá-la com `library(...)`:

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.1.4      v readr      2.1.5  
v forcats    1.0.0      v stringr    1.5.1  
v ggplot2    3.5.2      v tibble     3.3.0  
v lubridate  1.9.4      v tidyr      1.3.1  
v purrr      1.0.4  
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

O tidyverse é uma coleção de pacotes para ciência de dados que compartilham uma filosofia de design.

Ele inclui pacotes para manipulação de dados (`dplyr`, `tidyr`), visualização (`ggplot2`) e muito mais.

Suas funções para manipulação de data frames (aqui chamados de tibbles), como `filter`, `select` e `mutate`, facilitam a análise de dados tabulares:

```
alunos <- tibble(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  nota_final = c(8.5, 9.0, 6.5),
  turma = c("A", "B", "A")
)

alunos_filtrados <- alunos |>
  filter(idade > 25) |> # filtrar os alunos com mais de 25 anos
  mutate(aprovado = nota_final >= 7) |> # nova coluna "aprovado" com base na nota final
  select(nome, idade, aprovado) # selecionar apenas as colunas nome e idade

print(alunos_filtrados)
```

```
# A tibble: 2 x 3
  nome  idade aprovado
<chr> <dbl> <lgl>
1 João    30 TRUE
2 Pedro   28 FALSE
```

Calculando a média de idade dos alunos aprovados:

```
media_idade_aprovados <- alunos |>
  filter(nota_final >= 7) |>
  summarise(media_idade = mean(idade))

print(media_idade_aprovados)
```

```
# A tibble: 1 x 1
  media_idade
<dbl>
1      27.5
```

Em resumo, vimos exemplos de funções:

- `filter()`: filtra linhas com base em condições lógicas
- `mutate()`: cria novas colunas ou modifica colunas existentes
- `select()`: seleciona colunas específicas
- `summarise()`: calcula estatísticas resumidas, como média, soma, etc

Existem várias outras funções para manipulação de data frames incluídas no **tidyverse**. Um resumo visual dessas funções está disponível em <https://nyu-cdsc.github.io/learningr/assets/data-transformation.pdf>.

1.3.9 Strings (cadeias de caracteres)

String é o tipo de dado utilizado para representar textos. Em R, strings são definidas com aspas simples ou duplas:

```
texto <- "Olá, mundo!"  
  
print(texto)
```

```
[1] "Olá, mundo!"
```

A biblioteca **stringr** (parte do **tidyverse**) possui diversas funções para a manipulação de strings (todas começam com o prefixo **str_**):

```
a <- "olá"  
b <- 'mundo'  
a_b_concatenados <- str_c(a, b, sep = " ")  
  
print(a_b_concatenados)
```

```
[1] "olá mundo"
```

- Concatenar significa unir duas ou mais strings. A função `str_c` do pacote **stringr** é usada para esta finalidade, e o argumento **sep** especifica o caractere separador a ser inserido entre as strings (neste caso, um espaço).

```
num_palavras <- str_length("quantos caracteres tem esta frase?")  
  
print(num_palavras)
```

[1] 34

- A função `str_length` retorna o número de caracteres em uma string, incluindo espaços e pontuação.

```
contem_texto <- str_detect("este texto contém palavras", "texto")  
  
print(contem_texto)
```

[1] TRUE

- A função `str_detect` verifica se uma string contém um padrão de texto. Ela retorna TRUE se o padrão for encontrado e FALSE caso contrário.

As funções do pacote `stringr` são vetorizadas, o que significa que elas podem ser aplicadas diretamente a um vetor de textos, retornando um resultado para cada elemento.

```
discursos <- c(  
  "este texto contém palavras",  
  "este outro texto não tem a palavra-chave",  
  "a palavra-chave está presente aqui"  
)  
contem_texto <- str_detect(discursos, "palavra-chave")  
  
print(contem_texto)
```

[1] FALSE TRUE TRUE

```
print(discursos[contem_texto])
```

```
[1] "este outro texto não tem a palavra-chave"  
[2] "a palavra-chave está presente aqui"
```

1.4 Exemplo: Análise de Sentimentos com R

```
library(tidyverse)
```

Criando um vetor de textos:

```

textos <- c(
  "Eu amo este produto, é incrível!",
  "Este serviço é péssimo, estou muito insatisfeito.",
  "O filme foi ótimo, adorei a história e os personagens.",
  "A comida estava horrível, não recomendo este restaurante.",
  "O atendimento foi excelente, muito atenciosos e rápidos.",
  "Não volto mais a este lugar."
)

print(textos)

```

```

[1] "Eu amo este produto, é incrível!"
[2] "Este serviço é péssimo, estou muito insatisfeito."
[3] "O filme foi ótimo, adorei a história e os personagens."
[4] "A comida estava horrível, não recomendo este restaurante."
[5] "O atendimento foi excelente, muito atenciosos e rápidos."
[6] "Não volto mais a este lugar."

```

Criando um data frame com os textos:

```

df_textos <- tibble(texto = textos)

print(df_textos)

```

```

# A tibble: 6 x 1
  texto
  <chr>
1 Eu amo este produto, é incrível!
2 Este serviço é péssimo, estou muito insatisfeito.
3 O filme foi ótimo, adorei a história e os personagens.
4 A comida estava horrível, não recomendo este restaurante.
5 O atendimento foi excelente, muito atenciosos e rápidos.
6 Não volto mais a este lugar.

```

Criando um vetor de palavras positivas e negativas:

```

palavras_positivas <- c("amo", "incrível", "ótimo",
                        "adorei", "excelente", "atenciosos", "rápidos")
palavras_negativas <- c("péssimo", "insatisfeito", "horrível",
                        "não recomendo", "ruim", "lento")

```

```
print(palavras_positivas)
```

```
[1] "amo"          "incrível"    "ótimo"       "adorei"      "excelente"
[6] "atenciosos" "rápidos"
```

```
print(palavras_negativas)
```

```
[1] "péssimo"      "insatisfeito" "horrível"     "não recomendo"
[5] "ruim"         "lento"
```

Criando uma função para analisar o sentimento de um texto:

```
analisar_sentimento <- function(palavras) {
  palavras_positivas_encontradas <- sum(palavras %in% palavras_positivas)
  palavras_negativas_encontradas <- sum(palavras %in% palavras_negativas)
  if (palavras_positivas_encontradas > palavras_negativas_encontradas) {
    return("Positivo")
  } else if (palavras_negativas_encontradas > palavras_positivas_encontradas) {
    return("Negativo")
  } else {
    return("Neutro")
  }
}

print(analisar_sentimento(c("este", "produto", "é", "incrível")))
```

```
[1] "Positivo"
```

Aplicando a função de análise de sentimento a cada texto:

```
df_analise <- df_textos |>
  mutate(texto_min = str_to_lower(texto)) |> # converter para minúsculas
  mutate(palavras = str_extract_all(texto_min, boundary("word")))|> # dividir o texto em palavras
  mutate(sentimento = map_chr(palavras, analisar_sentimento)) |> # map_chr aplica a função
  select(texto, sentimento) # selecionar apenas as colunas texto e sentimento

print(df_analise)
```



```
# A tibble: 6 x 2
  texto                                sentimento
  <chr>                                <chr>
1 Eu amo este produto, é incrível!      Positivo
2 Este serviço é péssimo, estou muito insatisfeito. Negativo
3 O filme foi ótimo, adorei a história e os personagens. Positivo
4 A comida estava horrível, não recomendo este restaurante. Negativo
5 O atendimento foi excelente, muito atenciosos e rápidos. Positivo
6 Não volto mais a este lugar.          Neutro
```

```
df_contagem <- df_analise |>
  group_by(sentimento) |> # agrupar por sentimento
  summarise(contagem = n()) # contar o número de textos em cada categoria de sentimento

print(df_contagem)
```

```
# A tibble: 3 x 2
  sentimento contagem
  <chr>          <int>
1 Negativo         2
2 Neutro           1
3 Positivo         3
```

1.5 Para saber mais

As três primeiras partes do livro *R for Data Science* (Wickham 2023) são uma ótima introdução à linguagem R e à manipulação de dados com a biblioteca `tidyverse`.

1.6 Atividade prática

1. Crie um vetor de textos contendo pelo menos 5 frases de discursos políticos.
2. Crie um data frame com esses textos.
3. Defina 3 vetores com palavras representando temas políticos (ex: economia, saúde, educação).
4. Crie uma função que analise um texto, identifique a qual tema ele mais se relaciona e retorne o tema predominante.
5. Aplique a função de análise de temas a cada texto e crie uma nova coluna no data frame indicando o tema predominante em cada texto.

2 Palavras, morfemas e tokens

i Nesta aula

- Otendo dados textuais
- Biblioteca *quanteda*
- Palavras, morfemas e tokens
- *Stopwords* (Palavras vazias)
- *Keyword-in-context* (Palavras em contexto)

Link para Google Colab: TODO

2.1 Obtendo dados textuais

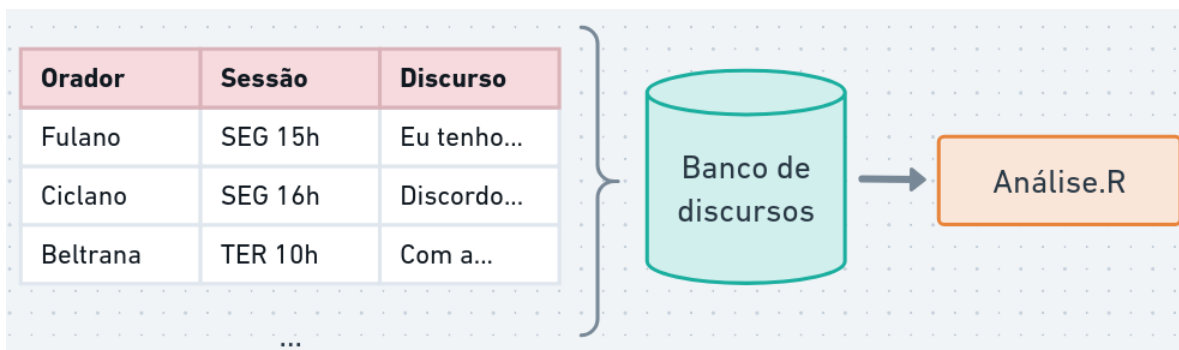


Quais são as fontes de textos interessantes para estudos políticos?

2.1.1 Tipos de fontes de dados textuais

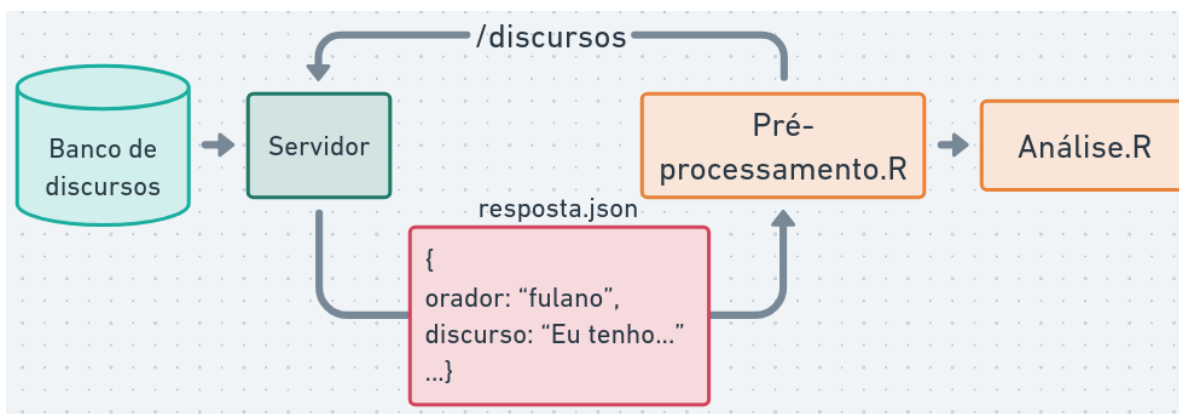
2.1.1.1 Banco de dados

- São coleções onde os textos já vêm organizados e prontos para análise.
- Eles costumam incluir “metadados”, que são informações valiosas de contexto sobre o texto (como a data, quem falou, o partido e o local).
- O acesso pode ser restrito por conta do alto custo de manutenção dessas plataformas.



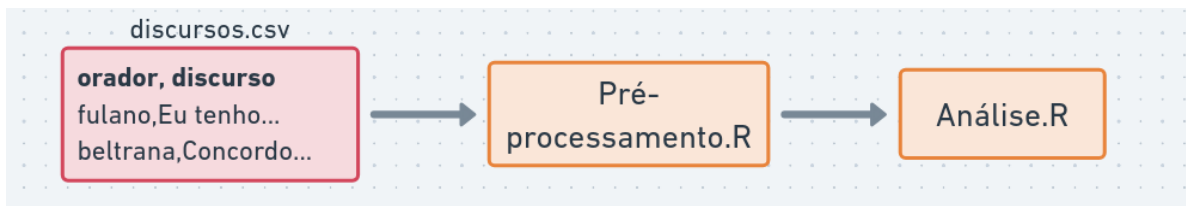
2.1.1.2 APIs (Interface de Programação de Aplicações)

- São ferramentas que nos permitem extrair dados de redes sociais e sites do governo (como o dados.gov.br) de forma automática.
- O acesso pode exigir um cadastro prévio ou ter um limite de buscas por dia.
- O formato dos dados varia para cada fonte (JSON ou XML).



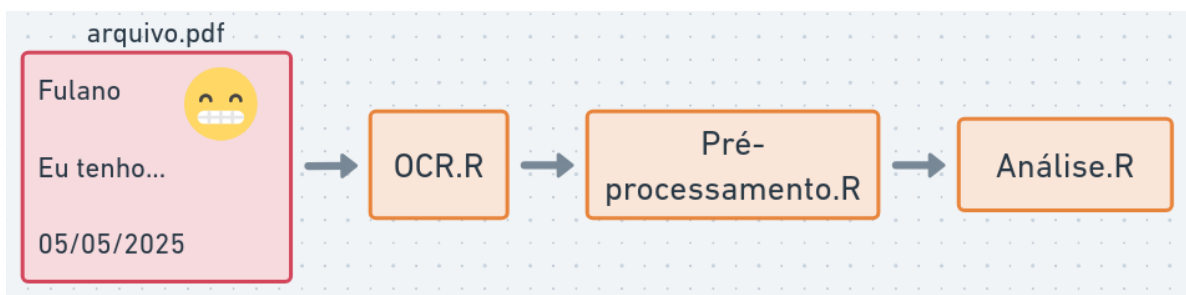
2.1.1.3 Arquivos de texto

- Formatos brutos (como planilhas CSV e arquivos TXT) que são muito fáceis de serem lidos pelo computador.
- No entanto, eles são estáticos e podem precisar de ajustes manuais de formatação.



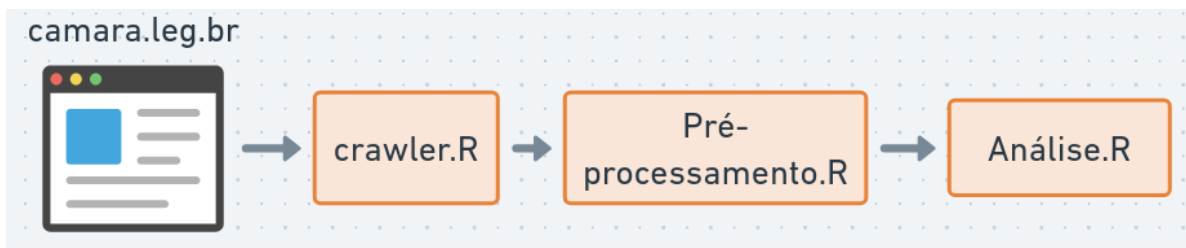
2.1.1.4 Arquivos não estruturados

- Documentos em PDF, Word, imagens e vídeos. O conteúdo de imagens e vídeos pode ser transformado em texto através de técnicas de transcrição automática.
- O processamento pode ser custoso e gerar ruídos.



2.1.1.5 Web scraping

- Extração de textos diretamente das páginas da internet utilizando ferramentas automatizadas.
- O formato dos dados muda de site para site e exige preparação prévia para organizar as informações.
- Se os criadores do site mudarem o layout da página, o processo de coleta pode parar de funcionar.



2.1.2 Exemplo: Obtendo Discursos da Câmara dos Deputados

A Câmara dos Deputados disponibiliza dados de discursos por meio de um sistema automatizado (API).

A documentação da API com exemplos está disponível em: <https://dadosabertos.camara.leg.br/>

Para baixar e organizar esses dados, utilizamos pacotes da linguagem R:

- `httr2` para fazer requisições na API
- `tidyverse` para organizar e tratar os dados

```
if (!"tidyverse" %in% installed.packages()) {  
  install.packages("tidyverse")  
}  
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.1.4      v readr      2.1.5  
v forcats    1.0.0      v stringr    1.5.1  
v ggplot2    3.5.2      v tibble     3.3.0  
v lubridate  1.9.4      v tidyr      1.3.1  
v purrr      1.0.4
```

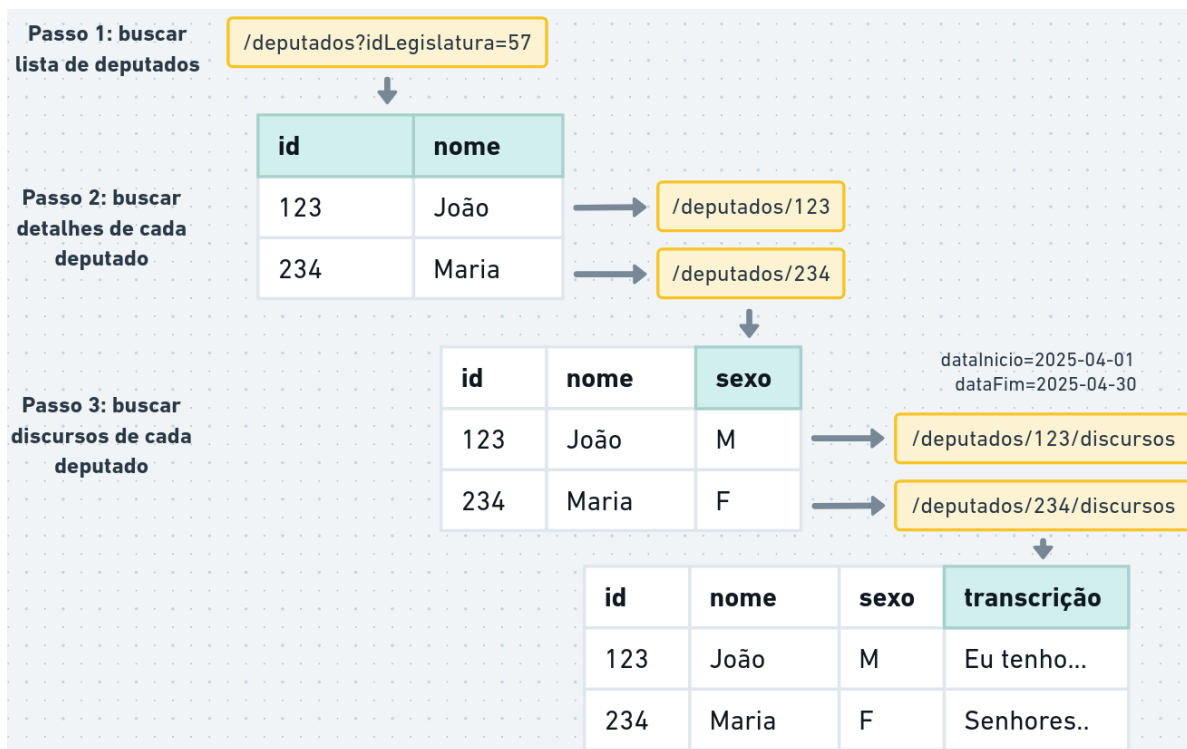
```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
if (!"httr2" %in% installed.packages()) {  
  install.packages("httr2")  
}  
library(httr2)
```



Primeiro, vamos montar uma requisição (função `request`) para obter dados dos deputados na legislatura 57 (atual):

```
url_api <- "https://dadosabertos.camara.leg.br/"

requisicao_deputados <- request(url_api) |>
  req_url_path("api", "v2", "deputados") |>
  req_url_query(
    idLegislatura = 57, # número da legislatura
    nome = "",         # parte do nome
    siglaUF = "",      # sigla UF eleito
    siglaPartido = "", # sigla do partido
    siglaSexo = "",    # sexo (M ou F)
  )

print(requisicao_deputados)
```

<httr2_request>

GET https://dadosabertos.camara.leg.br/api/v2/deputados?idLegislatura=57&nome=&siglaUF=&siglaPartido=&siglaSexo=

Body: empty

Executamos a requisição (`req_perform`) para obter a resposta do serviço de dados abertos:

```
resp_deputados <- req_perform(requisicao_deputados)

print(resp_deputados)
```

```
<httr2_response>
GET https://dadosabertos.camara.leg.br/api/v2/deputados?idLegislatura=57&nome=&siglaUF=&siglaPartido=
Status: 200 OK
Content-Type: application/json
Body: In memory (248328 bytes)
```

Os dados retornados estão no formato JSON, que é representado no R como uma lista aninhada (lista com elementos de lista).

```
dados_resposta <- resp_body_json(resp_deputados)
dados_resposta <- dados_resposta$dados # acessa elemento "dados" da lista

print(length(dados_resposta)) # número de elementos
```

```
[1] 722
```

```
print(dados_resposta[[1]]) # primeiro elemento
```

```
$id
```

```
[1] 220593
```

```
$uri
```

```
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/220593"
```

```
$nome
```

```
[1] "Abilio Brunini"
```

```
$siglaPartido
```

```
[1] "PL"
```

```
$uriPartido
```

```
[1] "https://dadosabertos.camara.leg.br/api/v2/partidos/37906"
```

```
$siglaUf
```

```
[1] "MT"
```

```
$idLegislatura
```

```
[1] 57
```

```
$urlFoto
```

```
[1] "https://www.camara.leg.br/internet/deputado/bandep/220593.jpg"
```

```
$email
```

```
NULL
```

- `$dados` acessa o elemento “dados” da lista, `[[1]]` acessa o primeiro elemento da lista.

Transformamos a lista em dataframe com o código comentado abaixo:

```
df_deputados_partido <- dados_resposta |>
  tibble() |>                                # transformar em dataframe
  unnest_wider("dados_resposta") |>         # desaninhar em colunas
  select(                                    # seleciona colunas relevantes
    id,
    nome,
    uf = siglaUf,
    partido = siglaPartido,
    uri
  )

head(df_deputados_partido) # head mostra as primeiras linhas do data frame
```

```
# A tibble: 6 x 5
```

	id	nome	uf	partido	uri
	<int>	<chr>	<chr>	<chr>	<chr>
1	220593	Abilio Brunini	MT	PL	https://dadosabertos.camara.leg.br/~
2	204379	Acácio Favacho	AP	MDB	https://dadosabertos.camara.leg.br/~
3	220714	Adail Filho	AM	REPUBLICANOS	https://dadosabertos.camara.leg.br/~
4	221328	Adilson Barroso	SP	PL	https://dadosabertos.camara.leg.br/~
5	204560	Adolfo Viana	BA	PSDB	https://dadosabertos.camara.leg.br/~
6	204528	Adriana Ventura	SP	NOVO	https://dadosabertos.camara.leg.br/~

Note que existem repetições, quando um deputado pertenceu a mais de um partido:


```
deputados_multiplos_partidos <- df_deputados_partido |>
  group_by(nome) |> # agrupar por nome
  filter(n() > 1)   # filtrar grupos com mais de um elemento

head(deputados_multiplos_partidos)
```

```
# A tibble: 6 x 5
# Groups:   nome [3]
      id nome                uf partido uri
  <int> <chr>                <chr> <chr> <chr>
1 220542 Alexandre Guimarães TO  REPUBLICANOS https://dadosabertos.camara.leg~
2 220542 Alexandre Guimarães TO  MDB          https://dadosabertos.camara.leg~
3 178881 Aluisio Mendes      MA   PSC          https://dadosabertos.camara.leg~
4 178881 Aluisio Mendes      MA   REPUBLICANOS https://dadosabertos.camara.leg~
5 220612 Bebeto              RJ   PTB          https://dadosabertos.camara.leg~
6 220612 Bebeto              RJ   PP           https://dadosabertos.camara.leg~
```

A coluna uri contém o endereço para buscarmos detalhes sobre o deputado, como sexo, data de nascimento, situação atual, etc.

Para cada deputado, vamos fazer a requisição que retorna os detalhes. Para isso, vamos mapear cada elemento da coluna uri em uma requisição:

```
df_deputados_detalhes <- df_deputados_partido |>
  mutate(req_detalhes = map(uri, request)) |> # montar requisições
  mutate(resp_detalhes = req_perform_parallel(req_detalhes)) # buscar detalhes e armazenar a
```

```
[working] (702 + 0) -> 10 -> 10 | >----- 1%
[working] (672 + 0) -> 10 -> 40 | ==>----- 6%
[working] (633 + 0) -> 10 -> 79 | ===>----- 11%
[working] (592 + 0) -> 11 -> 119 | =====>----- 16%
[working] (548 + 0) -> 10 -> 164 | =====>----- 23%
[working] (503 + 0) -> 10 -> 209 | =====>----- 29%
[working] (463 + 0) -> 10 -> 249 | =====>----- 34%
```

```

[working] (422 + 0) -> 10 -> 290 | =====>----- 40%

[working] (382 + 0) -> 10 -> 330 | =====>----- 46%

[working] (343 + 0) -> 10 -> 369 | =====>----- 51%

[working] (300 + 0) -> 10 -> 412 | =====>----- 57%

Waiting 28s for rate limit =>-----

Waiting 28s for rate limit ====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

[working] (300 + 0) -> 10 -> 412 | =====>----- 57%
[working] (244 + 1) -> 10 -> 468 | =====>----- 65%
Waiting 26s for rate limit =>-----
Waiting 26s for rate limit =>-----
Waiting 26s for rate limit =====>-----
Waiting 26s for rate limit =====>-----
Waiting 26s for rate limit =====>-----
Waiting 26s for rate limit =====>-----
Waiting 26s for rate limit =====>-----

```

```

Waiting 26s for rate limit =====>-----
Waiting 26s for rate limit =====>-----
Waiting 26s for rate limit =====>-----
Waiting 26s for rate limit =====>-----
[working] (244 + 1) -> 10 -> 468 | =====>----- 65%
[working] (185 + 2) -> 10 -> 527 | =====>----- 73%
[working] (146 + 2) -> 9 -> 567 | =====>----- 79%
[working] (105 + 2) -> 9 -> 608 | =====>----- 84%
[working] (63 + 2) -> 10 -> 649 | =====>----- 90%
[working] (27 + 2) -> 10 -> 685 | =====>----- 95%
[working] (0 + 2) -> 5 -> 717 | =====>----- 99%
[working] (0 + 2) -> 0 -> 722 | =====>----- 100%

```

```
head(df_deputados_detalhes)
```

```
# A tibble: 6 x 7
```

	id	nome	uf	partido	uri	req_detalhes	resp_detalhes
	<int>	<chr>	<chr>	<chr>	<chr>	<list>	<list>
1	220593	Abilio Brunini	MT	PL	https://~	<httr2_rq>	<httr2_rs>
2	204379	Acácio Favacho	AP	MDB	https://~	<httr2_rq>	<httr2_rs>
3	220714	Adail Filho	AM	REPUBLICANOS	https://~	<httr2_rq>	<httr2_rs>
4	221328	Adilson Barroso	SP	PL	https://~	<httr2_rq>	<httr2_rs>
5	204560	Adolfo Viana	BA	PSDB	https://~	<httr2_rq>	<httr2_rs>
6	204528	Adriana Ventura	SP	NOVO	https://~	<httr2_rq>	<httr2_rs>

- `req_perform_parallel` executa múltiplas requisições em paralelo (cada requisição da coluna `req_detalhes`).

Vamos extrair os detalhes da coluna `resp_detalhes` e organizar os dados na tabela.

```

df_deputados <- df_deputados_detalhes |>
mutate(detalhes = map(resp_detalhes, resp_body_json)) |> # ler dados da resposta
unnest_wider(detalhes) |> unnest_wider("dados", names_sep = "_") |> # desaninhar coluna "dados"
unnest_wider("dados_ultimoStatus", names_sep = "_") |> # desaninhar ultimoStatus
select( # selecionar colunas
  # colunas antigas
  colnames(df_deputados_partido),
  # novas colunas
  nome_civil = dados_nomeCivil,
  sexo = dados_sexo,
  cpf = dados_cpf,
  data_nascimento = dados_dataNascimento,

```

```

data_falecimento = dados_dataFalecimento,
uf_nascimento = dados_ufNascimento,
municipio_nascimento = dados_municipioNascimento,
escolaridade = dados_escolaridade,
situacao = dados_ultimoStatus_situacao,
data_situacao = dados_ultimoStatus_data,
ultimo_partido = dados_ultimoStatus_siglaPartido
) |>
filter(partido == ultimo_partido) |> # considerar apenas último partido
distinct()                          # filtrar dados duplicados

head(df_deputados)

```

```

# A tibble: 6 x 16
      id nome          uf partido uri nome_civil sexo cpf data_nascimento
  <int> <chr>         <chr> <chr> <chr> <chr> <chr> <chr> <chr>
1 220593 Abilio Brun~ MT PL http~ ABILIO JA~ M 9977~ 1984-01-31
2 204379 Acácio Fava~ AP MDB http~ ACÁCIO DA~ M 7428~ 1983-09-28
3 220714 Adail Filho AM REPUB~ http~ ADAIL JOS~ M 7726~ 1992-02-16
4 221328 Adilson Bar~ SP PL http~ ADILSON B~ M 0558~ 1964-06-14
5 204560 Adolfo Viana BA PSDB http~ ADOLFO VI~ M 8012~ 1981-02-02
6 204528 Adriana Ven~ SP NOVO http~ ADRIANA M~ F 1251~ 1969-03-06
# i 7 more variables: data_falecimento <chr>, uf_nascimento <chr>,
# municipio_nascimento <chr>, escolaridade <chr>, situacao <chr>,
# data_situacao <chr>, ultimo_partido <chr>

```

Para obter os discursos, vamos definir uma função que monta a requisição para um deputado (por número identificador):

```

req_discursos_deputado <- function(id_deputado) {
  req <- request(url_api) |>
  req_url_path("api", "v2", "deputados", id_deputado, "discursos") |>
  req_url_query(
    dataInicio = "2025-04-01",
    dataFim = "2025-04-30",
    itens=100
  )
  return(req)
}

print(req_discursos_deputado(160575))

```

```
<http2_request>
```

```
GET https://dadosabertos.camara.leg.br/api/v2/deputados/160575/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&itens=100
```

```
Body: empty
```

Da mesma forma que obtemos os dados dos deputados, vamos fazer requisições para obter discursos de cada deputado:

```
df_discursos <- df_deputados |>
  mutate(req_discursos = map(id, req_discursos_deputado)) |>      # montar a requisição chamada
  mutate(resp_discursos = req_perform_parallel(req_discursos)) |> # executar requisições em paralelo
  mutate(dados_discursos = map(resp_discursos, resp_body_json))    # ler dados das respostas
```

```
[working] (625 + 0) -> 8 -> 12 | =>----- 2%
```

```
[working] (587 + 0) -> 10 -> 48 | ==>----- 7%
```

```
[working] (544 + 0) -> 10 -> 91 | ===>----- 14%
```

```
[working] (500 + 0) -> 10 -> 135 | =====>----- 21%
```

```
[working] (456 + 0) -> 10 -> 179 | =====>----- 28%
```

```
[working] (419 + 0) -> 10 -> 216 | =====>----- 33%
```

```
[working] (374 + 0) -> 10 -> 261 | =====>----- 40%
```

```
[working] (332 + 0) -> 10 -> 303 | =====>----- 47%
```

```
[working] (289 + 0) -> 10 -> 346 | =====>----- 54%
```

```
[working] (246 + 0) -> 10 -> 389 | =====>----- 60%
```

```
[working] (207 + 0) -> 10 -> 428 | =====>----- 66%
```

```
Waiting 28s for rate limit =>-----
```

```
Waiting 28s for rate limit ==>-----
```

```

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>--

Waiting 28s for rate limit =====>

[working] (207 + 0) -> 10 -> 428 | =====>----- 66%
[working] (151 + 1) -> 10 -> 484 | =====>----- 75%
[working] (110 + 1) -> 11 -> 524 | =====>----- 81%
[working] (70 + 1) -> 10 -> 565 | =====>----- 88%
[working] (25 + 1) -> 10 -> 610 | =====>-- 95%
[working] (0 + 1) -> 0 -> 645 | =====> 100%

```

```
print(df_discurso$dados_discurso[[2]])
```

```

$dados
$dados[[1]]
$dados[[1]]$dataHoraInicio
[1] "2025-04-29T13:55"

$dados[[1]]$dataHoraFim
NULL

$dados[[1]]$uriEvento
[1] ""

$dados[[1]]$faseEvento

```

```
$dados[[1]]$faseEvento$titulo  
[1] "Encerramento"
```

```
$dados[[1]]$faseEvento$dataHoraInicio  
NULL
```

```
$dados[[1]]$faseEvento$dataHoraFim  
NULL
```

```
$dados[[1]]$tipoDiscurso  
[1] "DISCURSO ENCAMINHADO"
```

```
$dados[[1]]$urlTexto  
[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"
```

```
$dados[[1]]$urlAudio  
NULL
```

```
$dados[[1]]$urlVideo  
NULL
```

```
$dados[[1]]$keywords  
[1] "Deputado federal, Missão oficial, China, Balanço, Amapá, Fonte alternativa de energia, Desenvol"
```

```
$dados[[1]]$sumario  
[1] "O Deputado relatou sua participação em missão oficial à China, realizada entre 17 e 27 de"
```

```
$dados[[1]]$transcricao  
[1] "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAVACHO (SEM REGISTRO TAQUIGRÁFICO)"
```

```
$dados[[2]]  
$dados[[2]]$dataHoraInicio  
[1] "2025-04-02T10:32"
```

```
$dados[[2]]$dataHoraFim  
NULL
```

```
$dados[[2]]$uriEvento  
[1] ""
```

```
$dados[[2]]$faseEvento
```

\$dados[[2]]\$faseEvento\$titulo

[1] "Homenagem"

\$dados[[2]]\$faseEvento\$dataHoraInicio

NULL

\$dados[[2]]\$faseEvento\$dataHoraFim

NULL

\$dados[[2]]\$tipoDiscurso

[1] "HOMENAGEM"

\$dados[[2]]\$urlTexto

[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"

\$dados[[2]]\$urlAudio

NULL

\$dados[[2]]\$urlVideo

NULL

\$dados[[2]]\$keywords

[1] "Dia Nacional de Conscientização sobre o Autismo,Homenagem,Sessão solene"

\$dados[[2]]\$sumario

[1] "O Deputado discursou na sessão solene em homenagem ao Dia Mundial de Conscientização sol"

\$dados[[2]]\$transcricao

[1] "O SR. ACÁCIO FAVACHO (Bloco/MDB - AP. Sem revisão do orador.) - Presidente, bom dia. Ob
o para Brasília para dar a ele um tratamento adequado. E, no dia de hoje, ela apresentou ess

\$dados[[3]]

\$dados[[3]]\$dataHoraInicio

[1] "2025-04-01T13:55"

\$dados[[3]]\$dataHoraFim

NULL

\$dados[[3]]\$uriEvento

[1] ""


```

$dados[[3]]$faseEvento
$dados[[3]]$faseEvento$titulo
[1] "Encerramento"

$dados[[3]]$faseEvento$dataHoraInicio
NULL

$dados[[3]]$faseEvento$dataHoraFim
NULL

$dados[[3]]$tipoDiscurso
[1] "DISCURSO ENCAMINHADO"

$dados[[3]]$urlTexto
[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"

$dados[[3]]$urlAudio
NULL

$dados[[3]]$urlVideo
NULL

$dados[[3]]$keywords
[1] "Macapá (AP),Habitação,Política habitacional,Programa Minha Casa, Minha Vida (PMCMV),Min"

$dados[[3]]$sumario
[1] "O Deputado celebrou os avanços do Programa Casa Macapá, vinculado ao Programa Minha Casa"

$dados[[3]]$transcricao
[1] "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAVACHO (SEM REGISTRO TAQUIGRÁF"

$links
$links[[1]]
$links[[1]]$rel
[1] "self"

$links[[1]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&itens=100"

```

```

$links[[2]]
$links[[2]]$rel
[1] "first"

$links[[2]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&pagina=1&itens=100"

$links[[3]]
$links[[3]]$rel
[1] "last"

$links[[3]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&pagina=1&itens=100"

```

Por fim, vamos extrair dos dados em colunas:

```

df_discursos <- df_discursos |>
  unnest_wider(dados_discursos) |> # desaninhar resposta em colunas (dados e links)
  unnest(dados) |> # desaninhar dados de discursos em linha (um discurso por linha)
  unnest_wider(dados) |> # desaninhar dados em colunas
  unnest_wider(faseEvento, names_sep = "_") |> # desaninhar "faseEvento"
  select(
    # colunas anteriores
    colnames(df_deputados),
    # novas colunas
    hora_inicio = dataHoraInicio,
    fase_evento = faseEvento_titulo,
    tipo_discurso = tipoDiscurso,
    keywords,
    sumario,
    transcricao
  )
head(df_discursos)

```

A tibble: 6 x 22

id	nome	uf	partido	uri	nome_civil	sexo	cpf	data_nascimento
<int>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>

```

1 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
2 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
3 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
4 220714 Adail Filho  AM      REPUB~ http~ ADAIL JOS~ M      7726~ 1992-02-16
5 221328 Adilson Bar~ SP      PL       http~ ADILSON B~ M      0558~ 1964-06-14
6 221328 Adilson Bar~ SP      PL       http~ ADILSON B~ M      0558~ 1964-06-14
# i 13 more variables: data_falecimento <chr>, uf_nascimento <chr>,
#   municipio_nascimento <chr>, escolaridade <chr>, situacao <chr>,
#   data_situacao <chr>, ultimo_partido <chr>, hora_inicio <chr>,
#   fase_evento <chr>, tipo_discurso <chr>, keywords <chr>, sumario <chr>,
#   transcricao <chr>

```

Por fim, podemos extrair estatísticas sobre os oradores e salvar os dados em uma planilha CSV para usos futuros.

```
print(str_c("Número total de discursos: ", nrow(df_discursos)))
```

```
[1] "Número total de discursos: 1849"
```

```
print(str_c("Número total de oradores: ", df_discursos |> select(nome) |> n_distinct()))
```

```
[1] "Número total de oradores: 317"
```

```
print(str_c("Número total de partidos: ", df_discursos |> select(partido) |> n_distinct()))
```

```
[1] "Número total de partidos: 21"
```

```
print(str_c("Número total de estados: ", df_discursos |> select(uf) |> n_distinct()))
```

```
[1] "Número total de estados: 27"
```

Principais oradores:

```
df_discursos |>
  group_by(nome) |> # agrupar por nome do orador
  count(sort = TRUE) # contar e ordenar

```

```
# A tibble: 317 x 2
# Groups:   nome [317]
  nome          n
  <chr>        <int>
1 Cabo Gilberto Silva 57
2 Luiz Lima          48
3 Hildo Rocha        39
4 Marcel van Hattem   39
5 Bibi Nunes         36
6 Erika Kokay        36
7 Coronel Assis      30
8 Gilson Marques     30
9 Chico Alencar      29
10 Otoni de Paula     28
# i 307 more rows
```

Principais partidos:

```
df_discursos |>
  group_by(partido) |> # agrupar por partido do orador
  count(sort = TRUE)  # contar e ordenar
```

```
# A tibble: 21 x 2
# Groups:   partido [21]
  partido      n
  <chr>    <int>
1 PL      455
2 PT      373
3 NOVO    136
4 PSOL    129
5 UNIÃO   117
6 MDB     105
7 PSD     94
8 REPUBLICANOS 92
9 PSB     82
10 PCdoB   57
# i 11 more rows
```

Para facilitar novos processamentos, podemos salvar os dados em formato CSV:

```
write_csv(df_discursos, "discursos.csv")
```

Da mesma forma, podemos ler arquivos CSV utilizando a função `read_csv` do pacote `readr`, incluído na coleção `tidyverse`:

```
df_discursos <- read_csv("discursos.csv")
```

```
Rows: 1849 Columns: 22
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr  (17): nome, uf, partido, uri, nome_civil, sexo, cpf, uf_nascimento, mun...
```

```
dbl  (1): id
```

```
lgl  (1): data_falecimento
```

```
dtm  (1): hora_inicio
```

```
date (2): data_nascimento, data_situacao
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

2.1.2.1 Criando o corpus

Na análise de textos, chamamos de **corpus** o conjunto de textos que vamos estudar.

É a partir dele que os programas de computador identificam padrões na linguagem.

Quando temos várias coleções, utilizamos o termo **corpora**.

A biblioteca `quanteda` do R possui diversas funções para criar e manipular esse material.

```
if (!"quanteda" %in% installed.packages()) {  
  install.packages("quanteda")  
}  
library(quanteda)
```

```
Package version: 4.3.1
```

```
Unicode version: 15.0
```

```
ICU version: 72.1
```

```
Parallel computing: 12 of 12 threads used.
```

```
See https://quanteda.io for tutorials and examples.
```

O corpus é criado a partir de uma tabela (data frame) especificando o texto principal (`text`) e uma coluna (`doc_id`) como identificador único de cada texto:

```
num_discurso <- 1:nrow(df_discursos)

df_discursos <- df_discursos |>
  mutate(doc_id = str_c(num_discurso, nome, hora_inicio, sep = " ")) # concatenar "nome", "hora_inicio" e "num_discurso"

corpus_discursos <- corpus(df_discursos, text_field = "transcricao", docid_field = "doc_id")

print(corpus_discursos)
```

Corpus consisting of 1,849 documents and 21 docvars.

```
1 Acácio Favacho 2025-04-29 13:55:00 :
"DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAV..."

2 Acácio Favacho 2025-04-02 10:32:00 :
"O SR. ACÁCIO FAVACHO (Bloco/MDB - AP. Sem revisão do orador...."

3 Acácio Favacho 2025-04-01 13:55:00 :
"DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAV..."

4 Adail Filho 2025-04-02 14:44:00 :
"O SR. ADAIL FILHO (Bloco/REPUBLICANOS - AM. Pela ordem. Sem ..."

5 Adilson Barroso 2025-04-22 17:16:00 :
"O SR. ADILSON BARROSO (Bloco/PL - SP. Sem revisão do orador...."

6 Adilson Barroso 2025-04-09 16:00:00 :
"O SR. ADILSON BARROSO (Bloco/PL - SP. Sem revisão do orador...."

[ reached max_ndoc ... 1,843 more documents ]
```

As informações extras sobre os textos (os metadados, como nome do orador e partido) ficam salvas junto com o corpus e podem ser consultadas facilmente utilizando a função `docvars()`:

```
metadados <- corpus_discursos |>
  docvars() |> # obter metadados do corpus
  select(nome, partido, uf) # selecionar colunas de interesse

head(metadados)
```

	nome	partido	uf
1	Acácio Favacho	MDB	AP
2	Acácio Favacho	MDB	AP
3	Acácio Favacho	MDB	AP
4	Adail Filho	REPUBLICANOS	AM
5	Adilson Barroso	PL	SP
6	Adilson Barroso	PL	SP

Assim como `filter`, podemos obter subconjuntos de `corpus_discursos` com a função `corpus_subset()`:

```
corpus_discursos_pt <- corpus_discursos |>
  corpus_subset(partido == "PT")

print(corpus_discursos_pt)
```

```
Corpus consisting of 373 documents and 21 docvars.
39 Airtton Faleiro 2025-04-30 10:28:00 :
"O SR. AIRTON FALEIRO (Bloco/PT - PA. Sem revisão do orador.)..."

40 Airtton Faleiro 2025-04-29 16:52:00 :
"O SR. AIRTON FALEIRO (Bloco/PT - PA. Sem revisão do orador.)..."

41 Airtton Faleiro 2025-04-29 10:52:00 :
"O SR. AIRTON FALEIRO (Bloco/PT - PA) - Sr. Presidente, em no..."

42 Airtton Faleiro 2025-04-09 17:08:00 :
"O SR. AIRTON FALEIRO (Bloco/PT - PA. Pela ordem. Sem revisão..."

43 Airtton Faleiro 2025-04-01 16:24:00 :
"O SR. AIRTON FALEIRO (Bloco/PT - PA. Pela ordem. Sem revisão..."

57 Alencar Santana 2025-04-29 19:16:00 :
"O SR. ALENCAR SANTANA (Bloco/PT - SP. Sem revisão do orador...."

[ reached max_ndoc ... 367 more documents ]
```

Outras bibliotecas relacionadas que vamos utilizar são: `quanteda.textstats`, para gerar estatísticas a partir de `corpus`, e `quanteda.textplots`, para visualizações gráficas como núvens de palavras.

```

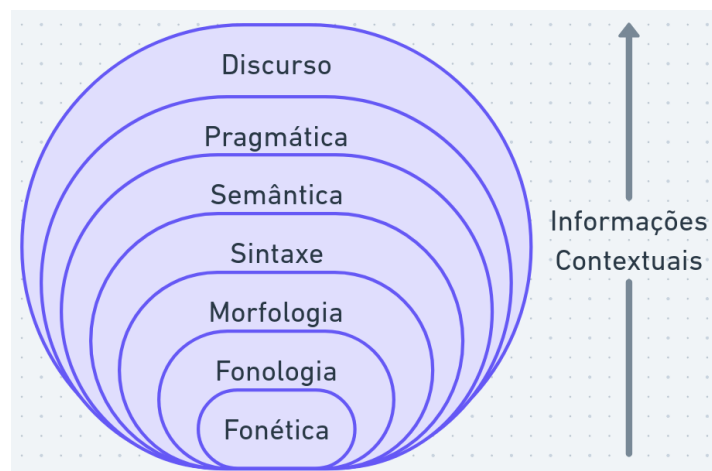
if (!"quanteda.textstats" %in% installed.packages()) {
  install.packages("quanteda.textstats")
}
library(quanteda.textstats)

if (!"quanteda.textplots" %in% installed.packages()) {
  install.packages("quanteda.textplots")
}
library(quanteda.textplots)

```

2.2 Níveis de análise textual

O estudo da língua é dividido em diversas subáreas:



- **fonética e fonologia:** sons e sua organização
- **morfologia:** como morfemas se organizam para formar palavras
- **sintaxe:** como palavras se organizam para formar sintagmas e orações
- **semântica:** significado de palavras e frases
- **pragmática:** organização de orações para fins comunicativos
- **discurso:** estruturas do texto como todo

O processamento de textos pelo computador foca principalmente no texto escrito (sintaxe, semântica e discurso).

Contudo, a fala e a entonação também são importantes, principalmente em estudos que envolvem áudio.

Além disso, dada a natureza multimodal da análise de discurso a fala também é relevante. Por exemplo:

<https://www.camara.leg.br/internet/sitaqweb/TextoHTML.asp?etapa=5&nuSessao=64.2025&nuQuarto=4327701&nuOrador=1&nuInsercao=1&dtHorarioQuarto=09:32&sgFaseSessao=BC&Data=30/04/2025&txApelido=Otoni%20de%20Paula&txFaseSessao=Breves%20Comunica%C3%A7%C3%B5es&txTipoSessao=Ordin%C3%A1ria%20-%20CD&dtHoraQuarto=09:32&txEtapa=>

Que tipo de informação é transmitida por meio da fala, além do conteúdo semântico?
Quais estão destacadas no texto taquigrafado?

2.3 A estrutura das palavras: morfemas, lemas e radicais

A forma como dividimos o texto depende do nível da nossa análise:

- partes de palavras: palavras relacionadas e significados
- palavras: análise sintática e semântica
- frases e parágrafos: entidades e modelos de discurso

Separar palavras em subunidades pode ser útil para:

- neologismos, palavras incorretas ou desconhecidas
- complexidade do texto
- regionalismos e dialetos antigos
- gêneros literários alternativos (poemas, músicas etc)

Por exemplo, Moffitt (2016) destaca o uso de neologismos em performances populistas (*refudiate* por Sarah Palin). Outros estudos comparam morfemas em diferentes estilos de texto, como jornalístico e literário.

2.3.1 Morfemas

Na morfologia, a unidade mínima de significado é o **morfema**:

- “casa” tem um morfema (casa)
- “casas” tem dois morfemas (casa + s)
- “casinha” tem dois morfemas (casa + inha)
- “casinhas” tem três morfemas (casa + inha + s)

A biblioteca `morphemepiece` possui funções para segmentação de palavras em morfemas da língua inglesa.

```
if (!"morphemepiece" %in% installed.packages()) {  
  install.packages("morphemepiece")  
}  
library(morphemepiece)
```

Exemplos de uso:

```
print(morphemepiece_tokenize("houses"))
```

```
[[1]]  
house   ##s  
5617   4034
```

```
print(morphemepiece_tokenize("orderly"))
```

```
[[1]]  
order   ##ly  
6478   4057
```

```
print(morphemepiece_tokenize("paradoxicity"))
```

```
[[1]]  
paradox ##icity  
9590   28877
```

```
print(morphemepiece_tokenize("discoursal"))
```

```
[[1]]  
discourse   ##al  
6456       4041
```

Existem ferramentas baseadas em dicionários e língua portuguesa do Brasil, por exemplo o projeto MorphoBR <https://github.com/LR-POR/MorphoBr>

```
echo "fortinho" | flockup -i morphobr.bin
fortinho      forte+N+DIM+M+SG
fortinho      forte+A+DIM+M+SG
```

- lema: forte
- N: nome
- A: adjetivo
- DIM: diminutivo
- M: masculino
- SG: singular

2.3.2 Stemização (Corte do radical)

A **stemização** é o processo de reduzir uma palavra ao seu radical (ou raiz).

Exemplos utilizando a biblioteca `quanteda`:

```
stems_testar <- char_wordstem(c("testar", "testando", "testarei"), language = "portuguese")
print(stems_testar)
```

```
[1] "test" "test" "test"
```

```
stems_love <- char_wordstem(c("love", "loving", "loved"), language = "english")
print(stems_love)
```

```
[1] "love" "love" "love"
```

Stemming pode ser útil para algumas tarefas de PLN, como análise de sentimentos ou classificação de texto.

Stemming vai reduzir o vocabulário do texto, facilitando também análises de frequência.

Porém, os radicais gerados pode gerar ambiguidades e perder informações importantes sobre o significado das palavras.

A **lematização** é o processo (mais preciso) de reduzir uma palavra à sua forma original de dicionário (ex: “gatos” e “gata” são convertidos para “gato”).

2.3.3 Caracteres

A menor unidade de informação em um texto digital é o **caractere** (ou símbolo).

O computador usa códigos, como o padrão Unicode, para entender e catalogar caracteres de todas as linguagens.

Ver: <>

A **codificação (encoding)** de textos se refere a como os caracteres são representados em números binários no computador (bytes).

O padrão mais comum é o **UTF-8** que utiliza tamanho variável de caracteres.

A biblioteca **quanteda** possui funções para separação de caracteres, como `tokenize_character`:

```
print(tokenize_character("avião = "))
```

```
[[1]]  
[1] "a"  "v"  "i"  "ã"  "o"  " "  "="  " "  " "  " "
```

```
print(tokenize_character("#feliz! \U0001f600"))
```

```
[[1]]  
[1] "#"  "f"  "e"  "l"  "i"  "z"  "!"  " "  " "
```

2.4 Tokenização (Dividindo o texto)

Não há consenso sobre a definição de *palavra* na linguística. Em PLN, a separação do texto em palavras depende do contexto da aplicação.

O problema de separar palavras (ou unidades mínimas de significado) em um texto é conhecido como **tokenização** e é uma etapa fundamental para muitas tarefas de PLN.

Note que o problema não é muito simples. O chinês, por exemplo, não utiliza espaço para separar palavras e cada caractere representa um morfema.

A biblioteca **quanteda** possui a função `tokens()` para realizar a tokenização de um texto, com diversas opções para personalizar o processo:

```
texto <- 'Ele disse: "Uh, são 13 horas. Não está na hora de almoçar?"'  
  
tokens_padrao <- tokens(texto)  
  
print(tokens_padrao)
```

Tokens consisting of 1 document.

```
text1 :  
 [1] "Ele"      "disse" ":"      "\""      "Uh"      ",,"      "são"      "13"      "horas"  
 [10] "."        "Não"      "está"  
 [ ... and 6 more ]
```

```
texto <- 'Ele disse: "Uh, são 13 horas. Não está na hora de almoçar?'"  
tokens <- tokens(texto, remove_punct = TRUE, remove_numbers = TRUE)  
print(tokens)
```

Tokens consisting of 1 document.

```
text1 :  
 [1] "Ele"      "disse"      "Uh"      "são"      "horas"      "Não"      "está"  
 [8] "na"      "hora"      "de"      "almoçar"
```

```
texto <- 'Ele disse: "Uh, são 13 horas. Não está na hora de almoçar?". Não, ela disse.'  
tokens <- tokens(texto, what = "sentence")  
print(tokens)
```

Tokens consisting of 1 document.

```
text1 :  
 [1] "Ele disse: \"Uh, são 13 horas.\" \"Não está na hora de almoçar?\".\""  
 [3] "Não, ela disse."
```

A tokenização em palavras é útil para análise sintática e semântica, enquanto a tokenização em sentenças é útil para análise de modelos de discurso.

```
tokens_discursos <- corpus_discursos |>  
  tokens(remove_punct = TRUE, remove_numbers = TRUE) |> # tokenizar texto  
  tokens_tolower()  
tokens_discursos
```

Tokens consisting of 1,849 documents and 21 docvars.

```
1 Acácio Favacho 2025-04-29 13:55:00 :  
 [1] "discurso"      "na"            "íntegra"      "encaminhado"  "pelo"
```

```

[6] "sr"          "deputado"      "acácio"          "favacho"        "sem"
[11] "registro"     "taquigráfico"
[ ... and 366 more ]

2 Acácio Favacho 2025-04-02 10:32:00 :
[1] "o"          "sr"          "acácio"        "favacho"        "bloco"
[6] "mdb"        "ap"          "sem"           "revisão"        "do"
[11] "orador"     "presidente"
[ ... and 531 more ]

3 Acácio Favacho 2025-04-01 13:55:00 :
[1] "discurso"    "na"          "íntegra"        "encaminhado"    "pelo"
[6] "sr"          "deputado"    "acácio"         "favacho"        "sem"
[11] "registro"    "taquigráfico"
[ ... and 497 more ]

4 Adail Filho 2025-04-02 14:44:00 :
[1] "o"          "sr"          "adail"          "filho"          "bloco"
[6] "republicanos" "am"          "pela"           "ordem"          "sem"
[11] "revisão"     "do"
[ ... and 204 more ]

5 Adilson Barroso 2025-04-22 17:16:00 :
[1] "o"          "sr"          "adilson"        "barroso"        "bloco"
[6] "pl"         "sp"          "sem"            "revisão"        "do"
[11] "orador"     "presidente"
[ ... and 405 more ]

6 Adilson Barroso 2025-04-09 16:00:00 :
[1] "o"          "sr"          "adilson"        "barroso"        "bloco"
[6] "pl"         "sp"          "sem"            "revisão"        "do"
[11] "orador"     "presidente"
[ ... and 384 more ]

[ reached max_ndoc ... 1,843 more documents ]

```

Em algumas aplicações, como modelos de linguagem, a tokenização é um processo essencial na construção do vocabulário. O **vocabulário** é a coleção de todas as palavras conhecidas (em geral, do texto ou do modelo).

Uma estrutura útil para análise de texto é a **matriz de documentos e termos (Document-Feature Matrix, DFM)**, que representa a frequência de cada palavra em cada documento:

```
dfm_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE) |> # tokenizar texto
  tokens_tolower() |> # converter palavras para minúsculo
  dfm() # criar matriz de documentos e termos

dfm_discursos
```

Document-feature matrix of: 1,849 documents, 25,831 features (99.29% sparse) and 21 docvars.

docs		features
		discurso na íntegra encaminhado pelo sr
1	Acácio Favacho 2025-04-29 13:55:00	1 4 1 1 1 1
2	Acácio Favacho 2025-04-02 10:32:00	0 2 0 0 0 3
3	Acácio Favacho 2025-04-01 13:55:00	1 2 1 1 3 1
4	Adail Filho 2025-04-02 14:44:00	0 2 0 0 1 2
5	Adilson Barroso 2025-04-22 17:16:00	0 3 0 0 0 1
6	Adilson Barroso 2025-04-09 16:00:00	0 5 0 0 0 1

docs		features
		deputado acácio favacho sem
1	Acácio Favacho 2025-04-29 13:55:00	2 1 1 1
2	Acácio Favacho 2025-04-02 10:32:00	1 1 1 1
3	Acácio Favacho 2025-04-01 13:55:00	1 1 1 1
4	Adail Filho 2025-04-02 14:44:00	0 0 0 1
5	Adilson Barroso 2025-04-22 17:16:00	0 0 0 1
6	Adilson Barroso 2025-04-09 16:00:00	2 0 0 2

[reached max_ndoc ... 1,843 more documents, reached max_nfeat ... 25,821 more features]

Cada linha é um documento e cada coluna é uma palavra. O valor em cada célula representa a frequência da palavra no documento.

```
print(dfm_discursos[6, "presidente"])
```

Document-feature matrix of: 1 document, 1 feature (0.00% sparse) and 21 docvars.

docs		features
		presidente
6	Adilson Barroso 2025-04-09 16:00:00	5

Por exemplo, para contar quantas vezes a palavra “presidente” é mencionada em todos os discursos, podemos somar os valores da coluna correspondente:

```
print(dfm_discursos[, "presidente"])
```

Document-feature matrix of: 1,849 documents, 1 feature (5.14% sparse) and 21 docvars.

docs		features
		presidente
1	Acácio Favacho 2025-04-29 13:55:00	2
2	Acácio Favacho 2025-04-02 10:32:00	4
3	Acácio Favacho 2025-04-01 13:55:00	1
4	Adail Filho 2025-04-02 14:44:00	1
5	Adilson Barroso 2025-04-22 17:16:00	3
6	Adilson Barroso 2025-04-09 16:00:00	5

[reached max_ndoc ... 1,843 more documents]

```
print(sum(dfm_discursos[, "presidente"]))
```

```
[1] 4737
```

2.5 Stopwords

Stopwords são palavras comuns que geralmente não carregam muito significado semântico, como “o”, “a”, “de”, “e” em português. Elas são frequentemente removidas em tarefas de PLN para reduzir o ruído e melhorar a eficiência do modelo.

Uma prática comum no processamento de textos é remover as stopwords antes de realizar análises, como contagem de palavras ou análise de sentimentos.

Podemos encontrar listas de stopwords em português e outras línguas na Internet, por exemplo, utilizando o pacote **stopwords**.

```
if (!"stopwords" %in% installed.packages()) {
  install.packages("stopwords")
}
library(stopwords)
```

```
stopwords_portugues <- stopwords(language = "pt")
print(stopwords_portugues[1:10])
```

```
[1] "de"    "a"     "o"     "que"   "e"     "do"    "da"    "em"    "um"    "para"
```


A remoção de stopwords pode não ser adequada para todas as tarefas, especialmente aquelas que dependem do contexto ou da estrutura do texto (análise sintática, reconhecimento de entidades, etc).

2.6 Palavras no contexto (Keywords-in-context)

Keywords-in-context (KWIC) é um método de indexação que busca uma palavra juntamente com seu contexto.

O **contexto** se refere às palavras próximas. O tamanho do contexto é o número de palavras antes e depois.

```
kwic_anistia <- corpus_discursos |>
  tokens() |>
  kwic("anistia") |>
  head(15)

kwic_anistia
```

Keyword-in-context with 15 matches.

```
[5 Adilson Barroso 2025-04-22 17:16:00, 88]
[22 Adriana Ventura 2025-04-10 11:32:00, 363]
[22 Adriana Ventura 2025-04-10 11:32:00, 374]
[39 Airton Faleiro 2025-04-30 10:28:00, 73]
[39 Airton Faleiro 2025-04-30 10:28:00, 395]
[39 Airton Faleiro 2025-04-30 10:28:00, 619]
[39 Airton Faleiro 2025-04-30 10:28:00, 650]
[55 Alberto Fraga 2025-04-01 16:08:00, 361]
[60 Alencar Santana 2025-04-02 20:16:00, 1081]
[60 Alencar Santana 2025-04-02 20:16:00, 1105]
[62 Alencar Santana 2025-04-01 14:16:00, 24]
[62 Alencar Santana 2025-04-01 14:16:00, 158]
[62 Alencar Santana 2025-04-01 14:16:00, 228]
[62 Alencar Santana 2025-04-01 14:16:00, 271]
[66 Alexandre Lindenmeyer 2025-04-22 21:16:00, 59]
```

```
      seja aprovada a Lei da | Anistia |
      se posicionou a favor da | anistia |
nos posicionarmos a favor da | anistia |
      brasileira, que é a | anistia |
querem liberdade total, querem | anistia |
```

os autores principais não têm | anistia |
 se repita no Brasil. | Anistia |
 Nem vou falar aqui de | anistia |
 dizer" não" à | anistia |
 que temos que votar a | anistia |
 - Democracia sempre. Sem | anistia |
 ! Permitir qualquer tipo de | anistia |
 dizendo claramente:" Sem | anistia |
 . E querem aprovar a | anistia |
 gritos, manifestações múltiplas sobre | anistia |

, a extrema esquerda vai
 . E a razão de
 não é passar a mão
 . Eu, sinceramente,
 total. Eu acho até
 e nem têm que ter
 , não! Mas,
 , porque sou do tempo
 . Alguns setores que obstruíram
 . Nós não podemos permitir
 . Prisão para os golpistas
 a eles significa, Deputado
 !" Prisão àqueles golpistas
 previamente porque sabem que têm
 , sobre o que aconteceu

- ser ve para analisar termos específicos e seu significado no texto
- é útil para analisar marcadores de discurso (ex: porque, assim, portanto, etc) e identificar estratégias e intensões
- desvantagem: o tamanho do contexto é limitado

```
golp_kwic <- corpus_discursos |>
  tokens() |>
  kwic("golp*", window = 3) |>
  head(15)

golp_kwic
```

Keyword-in-context with 15 matches.

[39 Airtton Faleiro 2025-04-30 10:28:00, 255] quem idealizou o |

[39 Airtton Faleiro 2025-04-30 10:28:00, 262]	a mobilização do
[39 Airtton Faleiro 2025-04-30 10:28:00, 352]	, orquestrou o
[39 Airtton Faleiro 2025-04-30 10:28:00, 444]	quem executou o
[39 Airtton Faleiro 2025-04-30 10:28:00, 474]	impedir que esse
[39 Airtton Faleiro 2025-04-30 10:28:00, 534]	, executa um
[39 Airtton Faleiro 2025-04-30 10:28:00, 562]	tentar dar um
[44 Alberto Fraga 2025-04-29 19:20:00, 124]	preparando mais um
[62 Alencar Santana 2025-04-01 14:16:00, 29]	Prisão para os
[62 Alencar Santana 2025-04-01 14:16:00, 52]	Brasil sofreu um
[62 Alencar Santana 2025-04-01 14:16:00, 138]	tentaram praticar o
[62 Alencar Santana 2025-04-01 14:16:00, 233]	" Prisão àqueles
[62 Alencar Santana 2025-04-01 14:16:00, 243]	, lideraram aquele
[66 Alexandre Lindenmeyer 2025-04-22 21:16:00, 367]	final do decreto
[75 Alice Portugal 2025-04-29 17:00:00, 563]	todo tipo de

golpe	, quem financiou
golpe	, quem praticou
golpe	contra a democracia
golpe	contra a democracia
golpe	se efetivasse,
golpe	de Estado,
golpe	, ocupar e
golpe	. Isso aqui
golpistas	. É isso
golpe	, praticado pelos
golpe	no dia 8
golpistas	que participaram,
golpe	, aquela tentativa
golpista	e pressionou militares
golpe	, como o

```
kwic_defendeu_acusou <- corpus_discursos |>
  tokens() |>
  kwic(c("portanto", "dessa forma", "assim", "porém")) |>
  head(15)

kwic_defendeu_acusou
```

Keyword-in-context with 15 matches.

[2 Acácio Favacho 2025-04-02 10:32:00, 632]	precisam ser atendidas. É
[5 Adilson Barroso 2025-04-22 17:16:00, 359]	funcionários dos Correios; e
[5 Adilson Barroso 2025-04-22 17:16:00, 362]	; e assim sucessivamente.

[13 Adriana Ventura 2025-04-15 18:20:00, 815]	os combinados secretos. É
[24 Adriana Ventura 2025-04-09 20:44:00, 123]	registrando a minha reclamação,
[25 Adriana Ventura 2025-04-01 17:24:00, 141]	a questão da imprevisibilidade.
[28 Afonso Hamm 2025-04-23 14:16:00, 264]	tragédia do ano passado.
[29 Afonso Hamm 2025-04-22 14:28:00, 230]	um socorro muito especial.
[31 Afonso Hamm 2025-04-01 14:32:00, 355]	até o carnaval".
[31 Afonso Hamm 2025-04-01 14:32:00, 412]	% do carvão nacional.
[31 Afonso Hamm 2025-04-01 14:32:00, 489]	no Município de Candiota.
[34 Afonso Motta 2025-04-22 19:12:00, 65]	respeito ao tema tratado.
[43 Airton Faleiro 2025-04-01 16:24:00, 46]	de Garrafão do Norte.
[44 Alberto Fraga 2025-04-29 19:20:00, 93]	Dilma a ser cassada.
[45 Alberto Fraga 2025-04-29 17:44:00, 133]	foi isso que disseram.

assim		que vamos conseguir avançar em
assim		sucessivamente. Portanto, este
Portanto		, este é o conselho
assim		: um faz uma coisa
porém		nós somos a favor dessa
Assim		, na minha visão,
Portanto		, peço socorro ao agro
Portanto		, essa audiência pública acontecerá
Assim		se passou todo esse tempo
Portanto		, Ministro de Minas e
Portanto		, eu encaminhei um projeto
Portanto		, orientamos" não"
Assim		como eles, Prefeitos e
Portanto		, a gente sabe que
Portanto		, a CBF precisa urgentemente

```
kwic_nomes <- corpus_discursos |>
  head(n = 50) |>
  tokens() |>
  kwic(phrase("Deputad[ao] [A-Z][a-z]+ [A-Z][a-z]+"), valuetype = "regex", case_insensitive = FALSE)

kwic_nomes
```

Keyword-in-context with 22 matches.

[1 Acácio Favacho 2025-04-29 13:55:00, 97:99]	nosso partido o MDB,
[6 Adilson Barroso 2025-04-09 16:00:00, 455:457]	contra. O voto do
[6 Adilson Barroso 2025-04-09 16:00:00, 482:484]	abraço do seu amigo o
[9 Adriana Ventura 2025-04-29 17:44:00, 44:46]	de Dança de Joinville o
[10 Adriana Ventura 2025-04-23 19:40:00, 53:55]	membro da nossa bancada o

[10 Adriana Ventura 2025-04-23 19:40:00, 75:77] os mesmos valores. O
[10 Adriana Ventura 2025-04-23 19:40:00, 161:163] falando também em nome do
[10 Adriana Ventura 2025-04-23 19:40:00, 187:189] Estou falando em nome do
[10 Adriana Ventura 2025-04-23 19:40:00, 267:269] princípios que nos elegeram.
[10 Adriana Ventura 2025-04-23 19:40:00, 319:321] da bancada. Obrigada,
[11 Adriana Ventura 2025-04-16 11:16:00, 50:52] , sob o comando do
[12 Adriana Ventura 2025-04-16 10:48:00, 269:271] faltando no nosso País,
[14 Adriana Ventura 2025-04-14 19:32:00, 75:77] especial ao Relator, o
[14 Adriana Ventura 2025-04-14 19:32:00, 86:88] Relator ad hoc, o
[14 Adriana Ventura 2025-04-14 19:32:00, 97:99] isso, e também ao
[16 Adriana Ventura 2025-04-14 18:52:00, 102:104] visão entre mim e o
[27 Afonso Hamm 2025-04-23 17:44:00, 112:114] e a organização do nosso
[28 Afonso Hamm 2025-04-23 14:16:00, 49:51] ontem, por iniciativa do
[28 Afonso Hamm 2025-04-23 14:16:00, 287:289] PSD - BA) -
[30 Afonso Hamm 2025-04-08 15:24:00, 131:133] importante para a base do
[39 Airtton Faleiro 2025-04-30 10:28:00, 25:27] Bom dia, Presidente,
[39 Airtton Faleiro 2025-04-30 10:28:00, 402:404] . Eu acho até,

Deputado Baleia Rossi	. Esta missão internacional teve
Deputado Adilson Barroso	é sempre para diminuir a
Deputado Federal Adilson	Barroso.
Deputado Coronel Armando	falou dele. Eu tenho
Deputado Luiz Lima	, que representa tudo que
Deputado Luiz Lima	é uma pessoa que luta
Deputado Ricardo Salles	, de tanta astúcia e
Deputado Gilson Marques	, nosso Deputado aguerrido de
Deputado Luiz Lima	, obrigada. Seja bem-vindo
Deputado Delegado Ramagem	e todos os outros.
Deputado Gilberto Abramo	e com toda a comunidade
Deputado Sargento Fahur	: respeito, respeito à
Deputado Gilberto Abramo	, ao nosso Relator ad
Deputado Coronel Assis	muito obrigada por isso,
Deputado Joaquim Passarinho	, que foi o Relator
Deputado Tarcísio Motta	, que me antecedeu e
Deputado Estadual Guilherme	Pasin. A beleza das
Deputado Afonso Hamm	, uma audiência pública exitosa
Deputado Afonso Hamm	, V.Exa. está inteiramente
Deputado Federal Afonso	Hamm. Anunciamos recursos para
Deputado Rogério Correia	, de Minas Gerais,
Deputado Rogério Correia	, que quem cometeu um

2.7 Exemplo: Palavras mais Utilizadas em Discursos

Primeiro, vamos tokenizar os discursos utilizando a função `tokens`:

```
tokens_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_symbols = TRUE, remove_numbers = TRUE)

tokens_discursos
```

Tokens consisting of 1,849 documents and 21 docvars.

```
1 Acácio Favacho 2025-04-29 13:55:00 :
  [1] "DISCURSO"      "NA"             "ÍTEGRA"         "ENCAMINHADO"    "PELO"
  [6] "SR"            "DEPUTADO"       "ACÁCIO"         "FAVACHO"        "SEM"
 [11] "REGISTRO"       "TAQUIGRÁFICO"
 [ ... and 366 more ]

2 Acácio Favacho 2025-04-02 10:32:00 :
  [1] "O"             "SR"             "ACÁCIO"         "FAVACHO"        "Bloco"
  [6] "MDB"           "AP"             "Sem"            "revisão"        "do"
 [11] "orador"        "Presidente"
 [ ... and 531 more ]

3 Acácio Favacho 2025-04-01 13:55:00 :
  [1] "DISCURSO"      "NA"             "ÍTEGRA"         "ENCAMINHADO"    "PELO"
  [6] "SR"            "DEPUTADO"       "ACÁCIO"         "FAVACHO"        "SEM"
 [11] "REGISTRO"       "TAQUIGRÁFICO"
 [ ... and 495 more ]

4 Adail Filho 2025-04-02 14:44:00 :
  [1] "O"             "SR"             "ADAIL"          "FILHO"          "Bloco"
  [6] "REPUBLICANOS" "AM"             "Pela"           "ordem"          "Sem"
 [11] "revisão"       "do"
 [ ... and 203 more ]

5 Adilson Barroso 2025-04-22 17:16:00 :
  [1] "O"             "SR"             "ADILSON"        "BARROSO"        "Bloco"
  [6] "PL"           "SP"             "Sem"            "revisão"        "do"
 [11] "orador"        "Presidente"
 [ ... and 405 more ]

6 Adilson Barroso 2025-04-09 16:00:00 :
  [1] "O"             "SR"             "ADILSON"        "BARROSO"        "Bloco"
```

```
[6] "PL"          "SP"          "Sem"          "revisão"      "do"
[11] "orador"      "Presidente"
[ ... and 384 more ]

[ reached max_ndoc ... 1,843 more documents ]
```

Agora vamos transformar todas as palavras em minúsculo e remover *stopwords* utilizando a função `tokens_remove`:

```
stopwords_portugues <- stopwords(language = "pt")

tokens_discursos <- tokens_discursos |>
  tokens_tolower() |>
  tokens_remove(stopwords_portugues)

print(tokens_discursos)
```

Tokens consisting of 1,849 documents and 21 docvars.

```
1 Acácio Favacho 2025-04-29 13:55:00 :
  [1] "discurso"      "íntegra"      "encaminhado"  "sr"           "deputado"
  [6] "acácio"        "favacho"      "registro"     "taquigráfico" "senhor"
 [11] "presidente"    "nobres"
 [ ... and 214 more ]

2 Acácio Favacho 2025-04-02 10:32:00 :
  [1] "sr"           "acácio"      "favacho"      "bloco"        "mdb"
  [6] "ap"           "revisão"     "orador"       "presidente"   "bom"
 [11] "dia"          "obrigado"
 [ ... and 291 more ]

3 Acácio Favacho 2025-04-01 13:55:00 :
  [1] "discurso"      "íntegra"      "encaminhado"  "sr"           "deputado"
  [6] "acácio"        "favacho"      "registro"     "taquigráfico" "senhor"
 [11] "presidente"    "senhoras"
 [ ... and 291 more ]

4 Adail Filho 2025-04-02 14:44:00 :
  [1] "sr"           "adail"       "filho"        "bloco"        "republicanos"
  [6] "am"           "ordem"       "revisão"      "orador"       "sr"
 [11] "presidente"    "sras"
 [ ... and 108 more ]
```

```

5 Adilson Barroso 2025-04-22 17:16:00 :
  [1] "sr"          "adilson"    "barroso"    "bloco"      "pl"
  [6] "sp"          "revisão"    "orador"     "presidente"  "colegas"
 [11] "deputados"  "todos"
 [ ... and 220 more ]

```

```

6 Adilson Barroso 2025-04-09 16:00:00 :
  [1] "sr"          "adilson"    "barroso"    "bloco"      "pl"
  [6] "sp"          "revisão"    "orador"     "presidente"  "colegas"
 [11] "venho"      "aqui"
 [ ... and 200 more ]

```

```
[ reached max_ndoc ... 1,843 more documents ]
```

Agora vamos construir a DFM:

```

dfm_discursos <- dfm(tokens_discursos)

print(dfm_discursos)

```

Document-feature matrix of: 1,849 documents, 25,653 features (99.45% sparse) and 21 docvars.

```

              features
docs          discurso íntegra encaminhado sr
1 Acácio Favacho 2025-04-29 13:55:00      1      1      1  1
2 Acácio Favacho 2025-04-02 10:32:00      0      0      0  3
3 Acácio Favacho 2025-04-01 13:55:00      1      1      1  1
4 Adail Filho 2025-04-02 14:44:00      0      0      0  2
5 Adilson Barroso 2025-04-22 17:16:00      0      0      0  1
6 Adilson Barroso 2025-04-09 16:00:00      0      0      0  1

              features
docs          deputado acácio favacho registro
1 Acácio Favacho 2025-04-29 13:55:00      2      1      1      1
2 Acácio Favacho 2025-04-02 10:32:00      1      1      1      0
3 Acácio Favacho 2025-04-01 13:55:00      1      1      1      1
4 Adail Filho 2025-04-02 14:44:00      0      0      0      0
5 Adilson Barroso 2025-04-22 17:16:00      0      0      0      0
6 Adilson Barroso 2025-04-09 16:00:00      2      0      0      0

              features
docs          taquigráfico senhor
1 Acácio Favacho 2025-04-29 13:55:00      1      1
2 Acácio Favacho 2025-04-02 10:32:00      0      0
3 Acácio Favacho 2025-04-01 13:55:00      1      1

```



```

4 Adail Filho 2025-04-02 14:44:00          0      0
5 Adilson Barroso 2025-04-22 17:16:00      0      0
6 Adilson Barroso 2025-04-09 16:00:00      0      0
[ reached max_ndoc ... 1,843 more documents, reached max_nfeat ... 25,643 more features ]

```

Para extrair algumas estatísticas básicas, como as palavras mais comuns, podemos utilizar a função `topfeatures`:

```

palavras_mais_comuns <- dfm_discursos |>
  textstat_frequency(n = 10)

palavras_mais_comuns

```

	feature	frequency	rank	docfreq	group
1	é	8970	1	1626	all
2	presidente	4737	2	1754	all
3	sr	3756	3	1687	all
4	aqui	2508	4	1043	all
5	brasil	2404	5	982	all
6	bloco	1863	6	1593	all
7	deputado	1862	7	811	all
8	revisão	1675	8	1661	all
9	governo	1606	9	640	all
10	hoje	1584	10	876	all

Podemos remover mais palavras que não colaboram para a interpretação dos dados, da mesma forma que removemos stopwords:

```

partidos <- docvars(dfm_discursos) |> select(partido) |> distinct() |> pull()
estados <- docvars(dfm_discursos) |> select(uf) |> distinct() |> pull()

palavras_irrelevantes <- c("senhor", "sr", "senhora", "sra", "sobre", "presidente", "deputado",
                           "senhora", "sra", "sobre", "presidente", "deputado")

dfm_discursos <- dfm_discursos |>
  dfm_remove(palavras_irrelevantes)

palavras_mais_comuns <- dfm_discursos |>
  textstat_frequency(n = 10)

palavras_mais_comuns

```

	feature	frequency	rank	docfreq	group
1	aqui	2508	1	1043	all
2	brasil	2404	2	982	all
3	revisão	1675	3	1661	all
4	governo	1606	4	640	all
5	hoje	1584	5	876	all
6	casa	1573	6	770	all
7	porque	1564	7	813	all
8	ser	1473	8	740	all
9	todos	1405	9	750	all
10	orador	1378	10	1364	all

```
suppressWarnings({ # desabilitar mensagens de aviso (warning)

dfm_discursos |>
  textplot_wordcloud()

})
```



```
suppressWarnings({ # desabilitar mensagens de aviso (warning)

dfm discursos |>
```


16	supremo	43	1	4	MISSÃO
17	governo	33	2	4	MISSÃO
18	gente	21	3	5	MISSÃO
19	vai	21	3	5	MISSÃO
20	agora	21	3	6	MISSÃO
21	aqui	218	1	88	NOVO
22	ser	151	2	73	NOVO
23	porque	137	3	73	NOVO
24	brasil	136	4	64	NOVO
25	governo	132	5	59	NOVO
26	brasil	76	1	29	PCdoB
27	aqui	71	2	30	PCdoB
28	quero	56	3	37	PCdoB
29	revisão	54	4	54	PCdoB
30	ordem	51	5	39	PCdoB
31	brasil	56	1	21	PDT
32	grande	45	2	19	PDT
33	aqui	44	3	16	PDT
34	casa	43	4	19	PDT
35	revisão	41	5	41	PDT
36	aqui	639	1	264	PL
37	brasil	605	2	235	PL
38	revisão	443	3	437	PL
39	governo	426	4	170	PL
40	porque	417	5	213	PL
41	brasil	99	1	23	PODE
42	aqui	93	2	21	PODE
43	todos	70	3	22	PODE
44	estado	53	4	17	PODE
45	hoje	47	5	18	PODE
46	estado	95	1	26	PP
47	aqui	66	2	29	PP
48	brasil	60	3	30	PP
49	quero	52	4	29	PP
50	revisão	51	5	50	PP
51	pessoas	21	1	5	PRD
52	brasil	20	2	6	PRD
53	muitas	19	3	6	PRD
54	todos	16	4	6	PRD
55	gente	16	4	3	PRD
56	acre	132	1	20	PSB
57	brasil	108	2	45	PSB
58	hoje	93	3	51	PSB

59	ser	88	4	32	PSB
60	federal	82	5	33	PSB
61	brasil	95	1	47	PSD
62	estado	91	2	32	PSD
63	aqui	88	3	42	PSD
64	cidade	86	4	25	PSD
65	obrigado	83	5	54	PSD
66	federal	55	1	12	PSDB
67	estado	36	2	11	PSDB
68	aqui	34	3	13	PSDB
69	sul	33	4	7	PSDB
70	governo	32	5	8	PSDB
71	aqui	213	1	87	PSOL
72	revisão	130	2	128	PSOL
73	glauber	129	3	34	PSOL
74	porque	125	4	63	PSOL
75	brasil	124	5	57	PSOL
76	aqui	558	1	224	PT
77	brasil	546	2	218	PT
78	governo	448	3	166	PT
79	porque	394	4	196	PT
80	casa	369	5	168	PT
81	governo	24	1	9	PV
82	brasil	23	2	7	PV
83	aqui	21	3	11	PV
84	federal	20	4	8	PV
85	casa	19	5	11	PV
86	cultura	13	1	2	REDE
87	partido	3	2	1	REDE
88	viva	3	2	1	REDE
89	além	2	4	1	REDE
90	povo	2	4	1	REDE
91	brasil	132	1	54	REPUBLICANOS
92	ser	92	2	34	REPUBLICANOS
93	aqui	90	3	46	REPUBLICANOS
94	todos	87	4	45	REPUBLICANOS
95	hoje	83	5	49	REPUBLICANOS
96	pessoas	18	1	3	SOLIDARIEDADE
97	autismo	11	2	1	SOLIDARIEDADE
98	revisão	9	3	9	SOLIDARIEDADE
99	ordem	8	4	8	SOLIDARIEDADE
100	orador	7	5	7	SOLIDARIEDADE
101	brasil	189	1	77	UNIÃO

102	aqui	153	2	63	UNIÃO
103	pessoas	151	3	45	UNIÃO
104	casa	143	4	56	UNIÃO
105	todos	124	5	59	UNIÃO

Para comparar as palavras mais comuns entre partidos, podemos utilizar a função `textplot_wordcloud` com o argumento `comparison = TRUE`:

```
suppressWarnings({ # desabilitar mensagens de aviso (warning)

dfm_discursos |>
  dfm_subset(partido %in% c("PT", "PL")) |>      # filtrar partidos PT e PL
  dfm_group(groups = partido) |>                # agrupar por partido
  textplot_wordcloud(comparison = TRUE, color = c("blue", "red"), min_count = 2) # gerar wordcloud

})
```



2.8 Para saber mais

Explore a documentação da API de dados abertos da Câmara dos Deputados: <https://www2.camara.leg.br/transparencia/dados-abertos/dados-abertos-legislativo/webservices/sessoesreunioes-2>.

Leia mais sobre morfemas, tokenização no Capítulo 2 do livro “Speech and Language Processing” (Jurafsky and Martin 2026) e os Capítulos 4 e 5 do livro “Processamento de Linguagem Natural” (Caseli and Nunes 2024).

Estude os tutoriais da biblioteca `quanteda` em <https://tutorials.quanteda.io/>. Saiba mais sobre a biblioteca lendo o artigo de Benoit et al. (2018).

2.9 Atividade prática

1. Obtenha os discursos do plenário da Câmara dos Deputados utilizando a API para um período de sua escolha.
2. Realize a tokenização e remova stopwords dos discursos.
3. Conte as palavras mais comuns e visualize os resultados utilizando uma nuvem de palavras (word cloud).
4. Compare as palavras mais comuns entre diferentes gêneros de oradores.
5. Utilize o argumento `groups` para construir o `dfm(..., groups = "partido_orador")` e `textplot_wordcloud(..., comparison = TRUE)` para gerar wordclouds comparativos.

Glossário

Introdução

ADP Análise de Discurso Político

PLN (NLP) Processamento de Linguagem Natural (Natural Language Processing)

Aula 1

R linguagem de programação utilizada para análise de dados

RStudio ambiente de desenvolvimento integrado (IDE) para R

Tidyverse coleção de pacotes R para ciência de dados

Variável nome para um valor armazenado em memória, que pode ser alterado

Condicional estrutura de código que executa ou não um bloco, de acordo com uma expressão lógica

Função um trecho de código que possui um nome, geralmente para uso de operações comuns

|> operador pipe utilizado para encadear chamadas de funções (o resultado é passado como argumento para a próxima função)

Vetor coleção ou sequência de um ou mais valores do mesmo tipo

Data Frame estrutura de dado para armazenar tabelas com linhas e colunas

Biblioteca coleção de funções, pode ser instalada e compartilhada em repositórios na Internet

String cadeia de caracteres, tipo de dado utilizado para representar texto

Referências

- Benoit, Kenneth, Kohei Watanabe, Haiyan Wang, Paul Nulty, Adam Obeng, Stefan Müller, and Akitaka Matsuo. 2018. “Quanteda: An r Package for the Quantitative Analysis of Textual Data.” *Journal of Open Source Software* 3 (30): 774. <https://doi.org/10.21105/joss.00774>.
- Caseli, H. M., and M. G. V. Nunes, eds. 2024. *Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações Em Português*. 2nd ed. BPLN. <https://brasileiraspln.com/livro-pln/2a-edicao/>.
- Jurafsky, Daniel, and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd ed. <https://web.stanford.edu/~jurafsky/slp3/>.
- Moffitt, Benjamin. 2016. *The Global Rise of Populism: Performance, Political Style, and Representation*. 1st ed. Stanford University Press. <https://doi.org/10.2307/j.ctvqsdsd8>.
- O’Neill, Lachlan, Nandini Anantharama, Wray Buntine, and Simon D. Angus. 2021. “Quantitative Discourse Analysis at Scale - AI, NLP and the Transformer Revolution.” *SoDa Laboratories Working Paper Series*, SoDa Laboratories Working Paper Series, December. <https://ideas.repec.org/p/ajr/sodwps/2021-12.html>.
- Silge, Julia, and David Robinson. 2017. *Text mining with R: a tidy approach*. Beijing, China: O’Reilly.
- Wickham, Hadley. 2023. *R for Data Science*. 2nd ed. Sebastopol: O’Reilly Media, Incorporated.

Expressões regulares

- Expressões regulares são padrões utilizados para buscar e manipular texto
- Permitem encontrar palavras, frases ou padrões específicos em um texto

A biblioteca `stringr` do R é uma das mais utilizadas para trabalhar com expressões regulares.

```
if (!"stringr" %in% installed.packages()) {  
  install.packages("stringr")  
}  
library(stringr)
```

A função `str_detect()` é utilizada para verificar se um padrão específico está presente em uma string.

Exemplo de expressão regular para encontrar textos que contém qualquer palavra iniciando com “brasil”:

```
discursos <- c(  
  "o Brasil é um país maravilhoso",  
  "a economia do Japão está crescendo",  
  "o futebol argentino é famoso",  
  "brasileiras são conhecidas por sua alegria",  
  "o clima no Brasil é tropical",  
  "brasileiros são apaixonados por futebol"  
)  
  
contem_brasil <- str_detect(discursos, "brasil.*")  
print(discursos[contem_brasil])
```

```
[1] "brasileiras são conhecidas por sua alegria"  
[2] "brasileiros são apaixonados por futebol"
```

- o ponto `.` significa “qualquer caractere” e o asterisco `*` significa “zero ou mais vezes”. Portanto, a expressão regular `brasil.*` corresponde a qualquer string que contenha a palavra “brasil” seguida de zero ou mais caracteres, ou seja, qualquer string que contenha a palavra “brasil” em qualquer posição.

Para encontrar o padrão “Brasil” (com B maiúsculo) ou “brasil” podemos utilizar:

```
contem_brasil <- str_detect(discursos, "brasil.*|Brasil.*")
print(discursos[contem_brasil])
```

```
[1] "o Brasil é um país maravilhoso"
[2] "brasileiras são conhecidas por sua alegria"
[3] "o clima no Brasil é tropical"
[4] "brasileiros são apaixonados por futebol"
```

<re1>|<re2> vai detectar a expressão re1 ou re2. Ou, de forma mais concisa:

```
contem_brasil <- str_detect(discursos, "[Bb]rasil.*")
print(discursos[contem_brasil])
```

```
[1] "o Brasil é um país maravilhoso"
[2] "brasileiras são conhecidas por sua alegria"
[3] "o clima no Brasil é tropical"
[4] "brasileiros são apaixonados por futebol"
```

- Os colchetes [] indicam um conjunto de caracteres. Portanto, a expressão regular [Bb]rasil.* corresponde a qualquer string que contenha “Brasil” com B maiúsculo ou “brasil” com b minúsculo, seguida de zero ou mais caracteres.

Podemos procurar por discursos que começam com “brasil” ou “Brasil” utilizando o símbolo ^ para indicar o início da string:

```
discursos <- c(
  "Brasil é um país maravilhoso",
  "a economia do Japão está crescendo",
  "o futebol argentino é famoso",
  "brasileiras são conhecidas por sua alegria",
  "o clima no Brasil é tropical",
  "brasileiros são apaixonados por futebol"
)

começam_com_brasil <- str_detect(discursos, "^[bB]rasil")
print(discursos[começam_com_brasil])
```

```
[1] "Brasil é um país maravilhoso"
[2] "brasileiras são conhecidas por sua alegria"
[3] "brasileiros são apaixonados por futebol"
```

De forma análoga o símbolo ` indica o final da string. Para encontrar discursos que terminam com “o”:

```
discursos <- c(
  "Brasil é um país maravilhoso",
  "a economia do Japão está crescendo",
  "o futebol argentino é famoso",
  "brasileiras são conhecidas por sua alegria",
  "o clima no Brasil é tropical",
  "brasileiros são apaixonados por futebol"
)

começam_com_brasil <- str_detect(discursos, "o$")
print(discursos[começam_com_brasil])
```

```
[1] "Brasil é um país maravilhoso"      "a economia do Japão está crescendo"
[3] "o futebol argentino é famoso"
```